

Solidum FEM

Manual de Arquitectura

Sigla: FF-MA

Visión arquitectural del programa: capas, bloques funcionales,
mecanismos transversales y dirección de evolución

Lectura del arquitecto, sin entrar al detalle de cada componente

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

19 de mayo de 2026

Índice general

Sobre este manual	IV
1. Filosofía del proyecto	1
1.1. Identidad del programa	1
1.2. Hipótesis fundacional	1
1.3. Modelo de colaboración	1
1.4. Principios rectores	2
1.5. Cuándo usar Solidum FEM y cuándo no	2
2. Estructura en capas	4
2.1. Mapa de capas	4
2.2. Responsabilidad de cada capa	4
2.3. Justificación de la separación	5
3. Flujo de un análisis	6
3.1. 1. Arranque	6
3.2. 2. Lectura del caso	6
3.3. 3. Numeración de grados de libertad	6
3.4. 4. Bucle del solver	7
3.5. 5. Cierre del paso	7
3.6. 6. Salida	7
3.7. Visión sintética	8
4. Bloques funcionales	9
4.1. Materiales	9
4.2. Elementos	10
4.2.1. Familia armadura (truss)	11
4.2.2. Familia cable	11
4.2.3. Familia marco / viga	12
4.2.4. Familia sólido 2D	13
4.2.5. Familia sólido 3D	13
4.2.6. Familias adicionales no implementadas	14
4.2.7. Contrato común y convención de signos	14
4.3. Solvers no lineales	14
4.4. Subsistema algebraico	16
4.5. Tipos de análisis	17

5. Mecanismos transversales	19
5.1. Registros con auto-registro	19
5.2. Descubrimiento automático	19
5.3. Contratos declarativos mediante atributos de clase	19
5.4. Validación temprana en construcción	20
5.5. Intérprete genérico por introspección de argumentos	20
5.6. Estados <i>trial</i> y comprometido en <code>ElementState</code>	20
5.7. Ensamblaje disperso con caché en formato <code>Coordinate</code>	20
5.8. Eliminación directa de condiciones de frontera	20
5.9. Compilación Just-In-Time mediante Numba	21
5.10. Variable principal de visualización	21
5.11. Cuadraturas centralizadas	21
5.12. Política unificada de tolerancias: patrón <code>atol + rtol · escala</code>	21
6. Convenciones de signos	23
6.1. Ejes y giros	23
6.2. Esfuerzos internos en 2D — convención de viga estructural	23
6.3. Esfuerzos internos en 3D — convención de resultantes de tensión bajo RHR	23
6.4. Capa interna y capa pública	24
7. Crecimiento del sistema	25
7.1. Tres puntos de extensión canónicos	25
7.2. Skill <code>/solidum-new</code>	25
7.3. Ciclo especificación → código → catálogo	26
7.4. Architecture Decision Records: memoria de arquitectura persistente	26
7.5. Mantenimiento de los documentos vivos	26
8. Decisiones de arquitectura (ADR)	27
8.1. ADR 0001 — Mecanismos de transparencia conceptual	27
8.2. ADR 0002 — Interfaz pública de resultados para consumidores externos	27
8.3. ADR 0003 — Despachador de algoritmo algebraico	28
8.4. ADR 0004 — Imposición de condiciones de Dirichlet por eliminación directa	29
8.5. ADR 0005 — Logging configurable en lugar de <code>print</code> directo	29
8.6. ADR 0006 — Tolerancias del criterio de admisibilidad constitutiva	30
8.7. ADR 0007 — Tolerancias de convergencia en solvers no lineales	30
8.8. ADR 0008 — Densidad como propiedad del material	31
8.9. ADR 0009 — Análisis modal y dinámico	31
8.10. Evolución de esta lista	33
9. Evolución de la arquitectura	34
9.1. Fase 1 (actual) — Estática lineal y no lineal	34
9.2. Fase 2 — Análisis modal	34
9.3. Fase 3 — Dinámica estructural transitoria lineal	35
9.4. Fase 4 — Dinámica estructural transitoria no lineal	36
9.5. Fase 4 (extensión 2026-05-18) — Cierre completo del subsistema modal/dinámico/es- pectral	37
9.6. Fase 5 — Problema térmico estacionario y transitorio	38
9.7. Fase 6 — Acoplamiento termo-mecánico	39

9.8. Mantenimiento de este capítulo	40
10. Compromisos de diseño	41
10.1. Paralelización con MPI	41
10.2. Adopción de PETSc o Trilinos como backend numérico	41
10.3. Arquitectura cliente-servidor o servicio web	42
10.4. Soporte de múltiples lenguajes en la interfaz pública	42
10.5. Sistema de plugins externo al repositorio	43
10.6. Refactor a tipado estático estricto con mypy <code>--strict</code>	43
10.7. Mantenimiento de este capítulo	44
11. Glosario	45
A. Pendientes de desarrollo	48
Bloques funcionales	48
Decisiones de arquitectura (ADR)	50
Evolución de la arquitectura	51

Sobre este manual

Este manual ofrece una **visión arquitectural** de Solidum FEM. Su objetivo es que el lector pueda reconstruir mentalmente cómo está organizado el programa, qué piezas tiene y cómo encajan, sin entrar al detalle de la formulación de cada elemento ni al algoritmo concreto de cada solver.

Para la *referencia formal* de cada componente (ecuaciones, matrices **B**, contratos YAML, criterios de aceptación), consultar `manuals/Reference_manual.pdf`, generado automáticamente desde `docs/specs/`.

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consultar `manuals/User_manual.pdf`.

Este manual se regenera con:

```
python manuals/build_architecture_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los archivos fuente en `manuals/sources/architecture/`.

Capítulo 1

Filosofía del proyecto

1.1 Identidad del programa

Solidum FEM es un programa de Método de Elementos Finitos (MEF) en aproximación de desplazamientos, orientado a la investigación en mecánica de sólidos: simulación de problemas mecánico y térmico, acoplados o desacoplados. Su público objetivo es la comunidad de investigación en mecánica computacional que requiere incorporar materiales, elementos o esquemas de solución sin verse limitada por la rigidez interna del programa.

1.2 Hipótesis fundacional

La mayor parte del coste de un programa de MEF de investigación no reside en la implementación del primer elemento, sino en el mantenimiento de un coste bajo para la incorporación del elemento $N+1$. Todo proyecto que crece sin disciplina de arquitectura acaba penalizando cada nueva incorporación: el primer material se implementa en una sesión; el décimo requiere modificar varios archivos; el vigésimo se desincentiva. Solidum FEM se diseña explícitamente para que la pendiente de coste de extensión se mantenga plana.

1.3 Modelo de colaboración

El desarrollo de Solidum FEM se rige por un contrato explícito entre el usuario (investigador con dominio profundo en MEF) y un asistente de inteligencia artificial responsable de la escritura del código. El flujo es: usuario $\rightarrow$ asistente $\rightarrow$ código $\rightarrow$ resultados $\rightarrow$ usuario. El asistente produce y modifica el código; el usuario cierra el lazo de validación sobre los resultados físicos y las decisiones de arquitectura.

La analogía operativa es la de una calculadora científica: se conoce qué representa la función seno, sus propiedades y el orden de magnitud esperado de su resultado, pero no se memoriza el algoritmo CORDIC interno; sí se detectan resultados absurdos. Trasladado al proyecto: la escritura del código (sintaxis, infraestructura, patrones de software) se delega; la comprensión conceptual no. El usuario mantiene un modelo mental completo del *qué* y del *por qué* de cada pieza de la arquitectura sin necesidad de escribir su implementación.

1.4 Principios rectores

Optimización para extensión. Cada decisión de arquitectura se evalúa también contra el coste de añadir el componente N+1. Un cambio que no abarata las extensiones futuras no se justifica únicamente por motivos estéticos. La consecuencia práctica es la preferencia por contratos declarativos sobre métodos abstractos cuando el comportamiento puede inferirse del dato, y por mecanismos de auto-registro sobre listas manuales de importaciones.

Equilibrio entre velocidad de cómputo y consumo de memoria. Las decisiones de implementación interna (estructuras dispersas, almacenamiento en formato Coordinate (COO) en caché, vectorización, compilación Just-In-Time (JIT)) buscan el compromiso entre tiempo de ejecución y consumo de memoria, y no la optimización aislada de uno solo de los dos ejes.

Mecanismos de transparencia conceptual. El sistema mantiene tres artefactos vivos para preservar la comprensión del usuario a medida que el código crece: un mapa de arquitectura (este manual y `docs/arquitectura.md`), un inventario explícito de los conceptos sostenedores (`docs/conceptos_clave.md`) y un protocolo conversacional para profundizar bajo demanda. Estos mecanismos se formalizan en el Architecture Decision Record (ADR) 0001.

Validación física mediante pruebas. Las sutilezas físicas (signos en notación de Voigt, factores de un medio, hipótesis de tensión plana frente a deformación plana) no se manifiestan en errores de compilación. Toda formulación nueva se acompaña de una prueba de validación contra solución analítica o frente a un caso de referencia conocido.

Convención de signos única. Toda formulación, prueba, interfaz pública y documentación utiliza la misma convención de signos. Si una referencia bibliográfica adopta otra, la traducción se realiza al implementar y se anota en la prueba correspondiente. Esta convención se detalla en el capítulo dedicado.

1.5 Cuándo usar Solidum FEM y cuándo no

La identidad de un programa se define tanto por su alcance como por sus límites. Esta sección delimita explícitamente lo que Solidum FEM pretende ser y lo que no.

Casos para los que Solidum FEM es la herramienta indicada. Investigación en mecánica de sólidos donde la prioridad es la **transparencia y la modificabilidad** del código: prototipado de modelos constitutivos nuevos antes de portarlos a un programa industrial; ensayo de variantes algorítmicas en solvers no lineales (predictores, correctores, criterios de longitud de arco); evaluación de formulaciones de elementos finitos contra soluciones analíticas y casos de referencia conocidos; trabajo docente y de tesis donde el estudiante debe comprender, modificar y validar cada pieza del cálculo.

Casos para los que Solidum FEM no es la herramienta indicada. Análisis de producción de piezas industriales con mallas de millones de elementos: el rendimiento absoluto y la paralelización masiva no son objetivos del proyecto. Análisis dinámicos transitorios o problemas multifísicos hoy: están previstos como evolución futura pero no implementados. Cálculos sometidos a normativa de certificación que requieren un programa con trazabilidad de validación industrial: se recomienda Code_Aster, Abaqus o ANSYS según el contexto. Modelado geomecánico avanzado o modelado de hormigón con plasticidad acoplada al daño: existen programas especializados (OpenSees, ATENA, DIANA) más maduros para estos dominios.

Posición frente a programas de referencia. Solidum FEM no compite en escala con los programas industriales: complementa el flujo de investigación. La hipótesis es que el código que el investigador puede leer, modificar y validar línea a línea es más valioso, en su contexto, que un

programa cerrado del que se obtiene un resultado sin posibilidad de auditoría interna. Cuando el modelo investigado madura y debe aplicarse a producción, la traducción a un programa industrial es la trayectoria natural.

Capítulo 2

Estructura en capas

El programa se organiza en seis capas con responsabilidades disjuntas. Cada capa se comunica con las contiguas mediante contratos explícitos; ninguna capa accede al interior de otra que no sea la inmediatamente inferior.

2.1 Mapa de capas

2.2 Responsabilidad de cada capa

Entrada del usuario. Un caso de Solidum FEM se describe de forma declarativa en un archivo en formato YAML (acrónimo recursivo de *YAML Ain't Markup Language*): nodos, materiales, elementos, condiciones de contorno, cargas y solver. La malla puede embeberse en el YAML o leerse desde un archivo `.msh` generado por Gmsh. El YAML constituye contrato público; cualquier consumidor externo (Interfaz Gráfica de Usuario o GUI, *scripts* de automatización) opera contra él.

Capa de inicialización. Su única responsabilidad consiste en poblar los registros (`MaterialRegistry`, `ElementRegistry`, `SolverRegistry`) sin que el resto del programa enumere manualmente las clases existentes. Esta tarea se ejecuta importando todos los módulos bajo carpetas canónicas; los decoradores `@register` se ejecutan en el momento de la importación e insertan cada clase en el registro correspondiente.

Capa de interpretación del caso. Traduce el archivo YAML en objetos. No contiene ramificaciones del tipo `if material_type == .Elastic1D` por cada material: introspecciona los argumentos formales del constructor de la clase recuperada del registro y le transmite los campos del YAML. La incorporación de un material nuevo no requiere modificación del intérprete.

Capa de dominio. El objeto `Domain` mantiene la numeración global de grados de libertad (DOF) y la conectividad. Las clases base `Element` y `Material` definen el contrato que cumple cada componente concreto. El objeto `ElementState` encapsula las dos copias de las variables internas (tensiones, variables de estado, historia) sobre las que opera el solver no lineal: una copia *trial* y otra comprometida.

Capa numérica. Concentra los componentes propios de la mecánica computacional: ensamblaje de matrices dispersas, cuadraturas de integración, estrategia de paso (lineal, no lineal incremental, longitud de arco) y el subsistema algebraico que resuelve los sistemas lineales que aparecen dentro de cada iteración.

Capa de salida. Provee dos productos. Por un lado, `VtkExporter` para visualización en programas compatibles con VTK (ParaView). Por otro, `SolveResult` con los desplazamientos, las

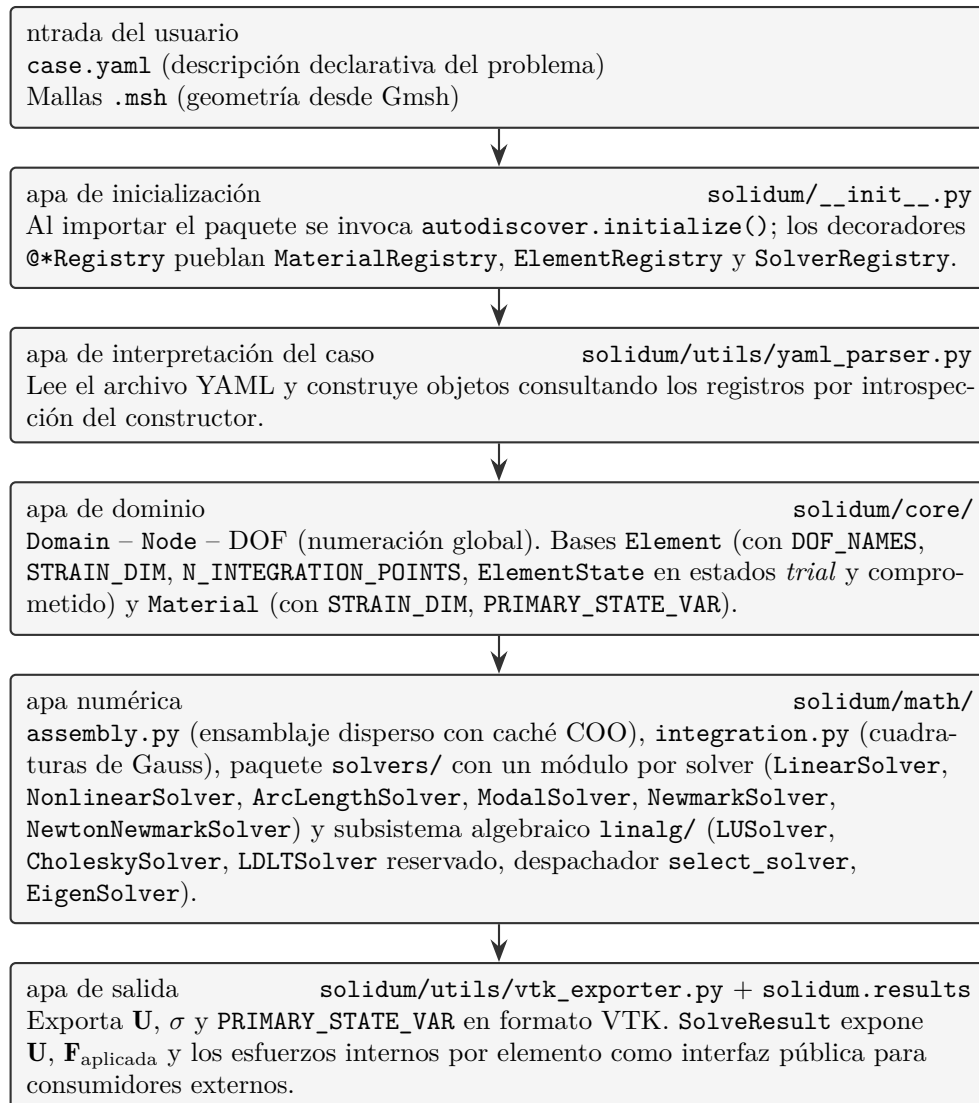


Figura 2.1: Estructura en capas de Solidum FEM. Cada capa se comunica con la inmediatamente inferior mediante contratos explícitos.

cargas aplicadas, las reacciones y los esfuerzos internos por elemento, concebido para consumidores externos como FenixBAR ([ADR 0002](#)).

2.3 Justificación de la separación

La frontera entre capas no es decorativa: define qué cambia con coste bajo y qué cambia con coste alto. La incorporación de un material nuevo afecta exclusivamente a la capa de dominio (`solidum/materials/<nombre>.py`) y no requiere modificar el intérprete del caso, el ensamblador ni el solver. La modificación del algoritmo del ensamblaje disperso no repercute sobre ningún material ni elemento concretos. La estanqueidad entre capas es el mecanismo que mantiene plana la pendiente de coste de extensión.

Capítulo 3

Flujo de un análisis

Esta sección describe, paso a paso, la secuencia de operaciones que se ejecuta sobre un caso desde su lanzamiento hasta la generación de resultados. La descripción es de alto nivel y omite detalles de implementación.

3.1 1. Arranque

El caso se lanza desde un *script* de usuario o desde la interfaz pública de entrada (`solidum.run` / `solidum.run_yaml`). Antes de que se abra el archivo YAML, se ejecuta la importación del paquete `solidum`. Dicha importación dispara el procedimiento de descubrimiento automático (*autodiscover*), que recorre las carpetas canónicas del proyecto e importa todos los módulos. Como cada material, elemento y solver lleva el decorador `@register` correspondiente, los registros quedan poblados sin enumeración manual.

3.2 2. Lectura del caso

El intérprete del caso abre el archivo YAML y construye los objetos del problema. Para cada material, elemento o solver mencionado en el archivo, se localiza su nombre en el registro correspondiente, se recupera la clase, se introspeccionan los argumentos requeridos por su constructor y se le transmiten los campos del YAML. El resultado es un grafo de objetos: nodos, materiales, elementos asociados a sus materiales, condiciones de contorno, cargas y un solver seleccionado.

En esta etapa se ejecuta la primera comprobación de consistencia: cuando un elemento se construye, su clase base verifica que el material recibido sea dimensionalmente compatible. Un elemento `Truss2D` con material `Elastic2D` produce un error en la fase de construcción del caso, no doscientas iteraciones después en forma de fallo opaco durante la solución.

3.3 3. Numeración de grados de libertad

El objeto `Domain` recorre nodos y elementos. Cada elemento declara los DOF requeridos en cada uno de sus nodos mediante el atributo `DOF_NAMES`. El dominio asigna índices globales de forma consistente con esa información y obtiene así el tamaño y la topología de la matriz de rigidez global. Esta numeración rige el ensamblaje durante toda la simulación.

3.4 4. Bucle del solver

El solver toma el control. Su comportamiento depende del tipo seleccionado (lineal, no lineal con Newton-Raphson, longitud de arco), pero todos los tipos comparten una estructura común: una secuencia de pasos de carga, una o varias iteraciones dentro de cada paso y, dentro de cada iteración, la siguiente secuencia interna:

1. Se solicita a cada elemento su matriz de rigidez tangente local `K_local` y su vector de fuerzas internas `f_internal`. El elemento las obtiene a partir del estado *trial* de sus puntos de Gauss y mediante una llamada al material con la deformación correspondiente.
2. El ensamblador construye la matriz global `K_global` y el vector global utilizando topología COO en caché: en la primera invocación se calculan los pares de índices; en las invocaciones siguientes se reescribe únicamente el vector de datos sobre la misma topología.
3. Se imponen las condiciones de Dirichlet por eliminación directa ([ADR 0004](#)). Toda restricción se expresa en forma afín $\mathbf{u}_s = \mathbf{g}_s + \sum \alpha_{si} \cdot \mathbf{u}_{mi}$ y se acumula en un `ConstraintSet`; el ensamblador produce el par $(\mathbf{T}, \mathbf{g})$ tal que $\mathbf{u} = \mathbf{T} \cdot \mathbf{u}_{libre} + \mathbf{g}$ y entrega al solver el sistema reducido $\mathbf{K}_{red} = \mathbf{T}^T \mathbf{K} \mathbf{T}$, $\mathbf{F}_{red} = \mathbf{T}^T (\mathbf{F} - \mathbf{K} \cdot \mathbf{g})$. La imposición es exacta a redondeo y preserva la simetría y la positividad de la matriz original.
4. El subsistema algebraico resuelve el sistema reducido $\mathbf{K}_{red} \cdot \delta \mathbf{U}_{libre} = \mathbf{F}_{red}$. Aquí interviene el despachador ([ADR 0003](#)): se inspeccionan las propiedades de `K_red` (simetría, posible definición positiva) y se selecciona el algoritmo numérico adecuado (factorización de Cholesky cuando aplica, factorización LU en el caso general). El ensamblador reconstruye después el incremento completo mediante `expand`.
5. Se evalúa la convergencia mediante un criterio dual: norma del residuo de fuerza y norma del incremento de desplazamientos, ambas comparadas contra una tolerancia de la forma `atol + rtol · escala(estado)` con la escala adaptativa al estado corriente del problema ([ADR 0007](#)). Los términos absolutos `atol` se autoderivan de las escalas del primer ensamblaje, lo que garantiza invariancia bajo cambio de unidades: el mismo problema planteado en N/m, kN/mm o MN/m converge en el mismo número de iteraciones hasta paridad de bits. El residuo de fuerza se evalúa únicamente sobre los DOF libres, dado que en los DOF prescritos el residuo contiene la reacción física, no un fallo de equilibrio. La convergencia del paso requiere que ambos criterios se cumplan simultáneamente (semántica AND, no OR).

3.5 5. Cierre del paso

Tras la convergencia de un paso, el solver invoca `commit_state()` sobre cada elemento. Esta operación promueve el estado *trial* a estado comprometido: las variables internas (deformaciones plásticas, daño, historia) que los puntos de Gauss han recorrido durante las iteraciones se consolidan. Si el paso no convergiera y se redujese el incremento, el estado *trial* se descartaría sin contaminar el estado comprometido.

3.6 6. Salida

A la finalización del solver, el objeto `Domain` construye un `SolveResult` inmutable que contiene los desplazamientos finales, las cargas aplicadas, las reacciones y los esfuerzos internos por elemento

en la convención de signos del proyecto. Este `SolveResult` constituye la interfaz pública (Application Programming Interface, API) que consume cualquier herramienta externa (GUI, *scripts* de post-proceso). De forma paralela, el módulo `VtkExporter` escribe un archivo VTK con los desplazamientos, las tensiones y la variable principal del material declarada por cada material mediante el atributo `PRIMARY_STATE_VAR`.

3.7 Visión sintética

A grandes rasgos, la secuencia es: importación → descubrimiento automático → interpretación del caso → numeración de DOF → bucle del solver ensamblaje → resolución algebraica → evaluación de convergencia → consolidación → salida. Cada flecha corresponde a un contrato declarado y verificado; cada paso intermedio es sustituible sin afectar al resto.

Capítulo 4

Bloques funcionales

Este capítulo describe los grandes bloques del programa desde el punto de vista funcional: la responsabilidad de cada uno, los componentes disponibles en la versión actual y el espacio de extensión natural. No se trata de un manual de referencia: para la formulación detallada de cada componente debe consultarse `manuals/Reference_manual.pdf` y los catálogos en `docs/`.

A lo largo del capítulo se utilizan las denominaciones abreviadas habituales para las dimensiones del espacio físico: una dimensión (1D), dos dimensiones (2D) y tres dimensiones (3D). En el contexto de los materiales, la **notación de Voigt** representa el tensor simétrico de deformación o tensión como un vector: con tres componentes para el caso bidimensional (Voigt-3) y con seis componentes para el caso tridimensional (Voigt-6). El atributo `STRAIN_DIM` de cada material indica el tamaño de ese vector y, por tanto, la dimensión geométrica del problema sobre el que opera.

4.1 Materiales

Un material en Solidum FEM es una ley constitutiva: dada una deformación (escalar para problemas 1D, en notación de Voigt-3 para problemas 2D, o en notación de Voigt-6 para problemas 3D, según el atributo `STRAIN_DIM`) y un estado interno, devuelve la tensión y el módulo tangente consistente para el solver. El material no posee información sobre el elemento que lo utiliza; el elemento no accede al interior del material. El acoplamiento se establece mediante el contrato declarativo `STRAIN_DIM` y la interfaz `compute_stress_and_tangent(strain, state)`.

Familias actualmente implementadas:

- *Elásticos lineales*: `Elastic1D` para deformación axial escalar; `Elastic2D` para problemas planos (tensión plana o deformación plana) en notación de Voigt 3.
- *Plásticos 1D*: `Elastoplastic1D` con criterio J2 axial, endurecimiento isótropo lineal y tangente algorítmica consistente.
- *Plásticos 2D — J2*: `VonMises2D` con criterio de fluencia J2 y algoritmo de retorno radial (*return mapping*) en notación de Voigt 3. Soporta dos hipótesis cinemáticas mutuamente excluyentes: deformación plana (Simó-Hughes §3.3) y tensión plana (*plane stress projected algorithm*, Simó-Hughes §3.4.1). Tangente algorítmica consistente cerrada en ambas.
- *Plásticos 2D — friccional*: `DruckerPrager2D` con criterio cohesivo-friccional (cono circular suave de Mohr-Coulomb), plasticidad no asociada por defecto y dos ramas cerradas de return (regular sobre el cono y al ápice). Tres calibraciones con Mohr-Coulomb (`plane_strain_matched`,

`outer_cone, inner_cone`). `IS_SYMMETRIC = False` cuando $\psi \neq \varphi$; el despachador algebraico (ADR 0003) degrada Cholesky→LU automáticamente.

- *Daño isotrópico*: `IsotropicDamage1D` y `IsotropicDamage2D`, daño escalar con softening exponencial. Tangente algorítmica consistente cerrada en carga activa (negativa en 1D durante softening; asimétrica en 2D porque $(C_e \cdot \varepsilon) - (M \cdot \varepsilon)$ no es simétrica). Secante en descarga.
- *Cables*: `CableMaterial1D`, ley axial con condición de tracción exclusiva (sin respuesta a compresión).

Espacio natural de extensión: cualquier ley constitutiva que pueda expresarse como una función $(\text{strain}, \text{state}) \rightarrow (\text{stress}, \text{tangent})$ se incorpora sin modificación del resto del programa. Candidatos directos:

- *[Pendiente] Modelo de Mohr-Coulomb 2D con esquinas (multi-superficie) para mayor fidelidad geotécnica respecto a Drucker-Prager.*
- *[Pendiente] Endurecimiento cinemático (Bauschinger) en J2 para plasticidad cíclica y fatiga.*
- *[Pendiente] Hiperelasticidad isotrópica (Neo-Hooke, Mooney-Rivlin, Ogden).*
- *[Pendiente] Viscoplasticidad de Perzyna y Duvaut-Lions.*
- *[Pendiente] Hiperelasticidad incompresible con tratamiento mixto desplazamiento-presión.*
- *[Pendiente] Drucker-Prager en tensión plana (la proyección con $\sigma_{zz}=0$ acoplada a flujo dilatante queda fuera del alcance de la versión actual).*

En todos los casos basta declarar el `STRAIN_DIM` apropiado y el atributo `PRIMARY_STATE_VAR` para visualización.

4.2 Elementos

Un elemento finito en Solidum FEM es una entidad geométrica que construye su matriz de gradientes `B`, su matriz de rigidez tangente y su vector de fuerzas internas a partir de los desplazamientos nodales y del material asociado. Cada elemento declara su `DOF_NAMES`, su `STRAIN_DIM` y el número de puntos de integración mediante `N_INTEGRATION_POINTS`. La clase base concentra la lógica común (registro de DOF, ensamblaje local→global, validación material↔elemento, gestión del `ElementState`) y cada subclase aporta exclusivamente lo físicamente específico (matriz `B`, esquema de integración, invocación al material).

El estado del programa por familia se presenta a continuación en forma de **matrices de cobertura**. Cada celda combina dos ejes ortogonales: el régimen geométrico (linealidad o no linealidad) y el régimen material (lineal o no lineal). La no linealidad material se obtiene mediante la asignación al elemento de un material no lineal del catálogo (plasticidad, daño); ningún elemento es intrínsecamente no lineal en el material. La marca ✓ indica combinación implementada y validada; la marca [Pendiente: ...] indica el hueco como candidato natural de extensión; el guion (—) indica combinación que no aplica físicamente o que no se contempla.

Tabla 4.1: Cobertura de la familia armadura — primer orden.

Régimen	2D · material lineal	2D · material no lineal	3D · material lineal	3D · material no lineal
Linealidad geométrica	✓ Truss2D	✓ Truss2D	✓ Truss3D	✓ Truss3D
No linealidad geométrica	✓ Truss2DCorot	✓ Truss2DCorot	✓ Truss3DCorot	✓ Truss3DCorot

4.2.1 Familia armadura (truss)

Elemento articulado que transmite exclusivamente esfuerzo axial. Material consumido: 1D escalar (Elastic1D, Elastoplastic1D, IsotropicDamage1D).

Primer orden (interpolación lineal, dos nodos)

Segundo orden (interpolación cuadrática, tres nodos)

Tabla 4.2: Cobertura de la familia armadura — segundo orden.

Régimen	2D · material lineal	2D · material no lineal	3D · material lineal	3D · material no lineal
Linealidad geométrica	<i>[Pendiente] Armadura 2D de segundo orden con linealidad geométrica.</i>	<i>[Pendiente] Armadura 2D de segundo orden con linealidad geométrica y material no lineal.</i>	<i>[Pendiente] Armadura 3D de segundo orden con linealidad geométrica.</i>	<i>[Pendiente] Armadura 3D de segundo orden con linealidad geométrica y material no lineal.</i>
No linealidad geométrica	<i>[Pendiente] Armadura 2D de segundo orden corrotacional.</i>	<i>[Pendiente] Armadura 2D de segundo orden corrotacional con material no lineal.</i>	<i>[Pendiente] Armadura 3D de segundo orden corrotacional.</i>	<i>[Pendiente] Armadura 3D de segundo orden corrotacional con material no lineal.</i>

4.2.2 Familia cable

Elemento de tracción exclusiva en formulación corrotacional. Material consumido: CableMaterial1D (sin respuesta a compresión).

Primer orden (interpolación lineal, dos nodos)

Tabla 4.3: Cobertura de la familia cable — primer orden.

Régimen	2D · material lineal a tracción	2D · material no lineal a tracción	3D · material lineal a tracción	3D · material no lineal a tracción
Linealidad geométrica	—	—	—	—
No linealidad geométrica	✓ Cable2DCorot	<i>[Pendiente] Cable 2D corrotacional con ley constitutiva no lineal a tracción (relajación, fluencia, daño tensional).</i>	✓ Cable3DCorot	<i>[Pendiente] Cable 3D corrotacional con ley constitutiva no lineal a tracción.</i>

La fila "Linealidad geom." se marca como no aplicable: un cable es, por su naturaleza física, un elemento de grandes desplazamientos (la rigidez axial es la única que aporta y depende de la configuración corriente).

Segundo orden (interpolación cuadrática, tres nodos)

Tabla 4.4: Cobertura de la familia cable — segundo orden.

Régimen	2D · material lineal	2D · material no lineal	3D · material lineal	3D · material no lineal
No linealidad geométrica	<i>[Pendiente] Cable 2D de segundo orden corrotacional.</i>	<i>[Pendiente] Cable 2D de segundo orden corrotacional con material no lineal.</i>	<i>[Pendiente] Cable 3D de segundo orden corrotacional.</i>	<i>[Pendiente] Cable 3D de segundo orden corrotacional con material no lineal.</i>

4.2.3 Familia marco / viga

Elemento con flexión y, en su caso, cortante y torsión. Material consumido: 1D escalar. La no linealidad material se introduce mediante el módulo tangente E_t del material, que escala uniformemente la sección; **la plasticidad distribuida en la sección (formulación fibra a fibra) no está implementada** y se marca como pendiente independiente.

Primer orden, formulación Euler-Bernoulli

Tabla 4.5: Cobertura de la familia marco — primer orden, formulación Euler-Bernoulli.

Régimen	2D · material lineal	2D · material no lineal (sección uniforme)	3D · material lineal	3D · material no lineal (sección uniforme)
Linealidad geométrica	✓ Frame2DEuler	✓ Frame2DEuler con Elastoplastic1D o IsotropicDamage1D	✓ Frame3D	✓ Frame3D con material 1D no lineal
No linealidad geométrica	✓ Frame2DEulerCorot	✓ Frame2DEulerCorot con material 1D no lineal	<i>[Pendiente] Marco 3D Euler-Bernoulli corrotacional con grandes rotaciones.</i>	<i>[Pendiente] Marco 3D Euler-Bernoulli corrotacional con material no lineal.</i>

Primer orden, formulación Timoshenko (con deformación por cortante)

Tabla 4.6: Cobertura de la familia marco — primer orden, formulación Timoshenko.

Régimen	2D · material lineal	2D · material no lineal (sección uniforme)	3D · material lineal	3D · material no lineal
Linealidad geométrica	✓ Frame2DTimoshenko	✓ Frame2DTimoshenko con material 1D no lineal	<i>[Pendiente] Marco 3D Timoshenko con deformación por cortante.</i>	<i>[Pendiente] Marco 3D Timoshenko con material no lineal.</i>
No linealidad geométrica	<i>[Pendiente] Marco 2D Timoshenko corrotacional.</i>	<i>[Pendiente] Marco 2D Timoshenko corrotacional con material no lineal.</i>	<i>[Pendiente] Marco 3D Timoshenko corrotacional.</i>	<i>[Pendiente] Marco 3D Timoshenko corrotacional con material no lineal.</i>

Segundo orden y formulaciones avanzadas

- *[Pendiente] Marco de segundo orden (interpolación cuadrática) en cualquiera de las formulaciones anteriores.*

- *[Pendiente] Plasticidad distribuida fibra a fibra en marcos 2D y 3D, con discretización transversal de la sección.*
- *[Pendiente] Marco con sección variable a lo largo del eje (sección no prismática).*

4.2.4 Familia sólido 2D

Elemento de medio continuo bidimensional bajo hipótesis de tensión plana o deformación plana. Material consumido: 2D en notación de Voigt 3 (Elastic2D, VonMises2D, IsotropicDamage2D, DruckerPrager2D).

Primer orden

Tabla 4.7: Cobertura de la familia sólido bidimensional — primer orden.

Topología	Material lineal	Material no lineal
Triangular lineal (3 nodos)	✓ Tri3	✓ Tri3 con VonMises2D, IsotropicDamage2D o DruckerPrager2D
Cuadrilátero bilineal (4 nodos)	✓ Quad4	✓ Quad4 con VonMises2D, IsotropicDamage2D o DruckerPrager2D

La columna de no linealidad geométrica se omite en esta familia: los elementos sólidos 2D actuales no incluyen formulación corrotacional ni lagrangiana actualizada.

- *[Pendiente] Sólido 2D con no linealidad geométrica (formulación lagrangiana actualizada o total).*

Segundo orden

Tabla 4.8: Cobertura de la familia sólido bidimensional — segundo orden.

Topología	Material lineal	Material no lineal
Triangular cuadrático P_2 (6 nodos)	✓ Tri6	✓ Tri6 con VonMises2D, IsotropicDamage2D o DruckerPrager2D
Cuadrilátero serendípito Q_2^{seren} (8 nodos)	✓ Quad8	✓ Quad8 con cualquier material 2D no lineal
Cuadrilátero lagrangiano Q_2 (9 nodos)	✓ Quad9	✓ Quad9 con cualquier material 2D no lineal

4.2.5 Familia sólido 3D

- *[Pendiente] Tetraedro lineal Tet4 (cuatro nodos), primer orden.*
- *[Pendiente] Tetraedro cuadrático Tet10 (diez nodos), segundo orden.*
- *[Pendiente] Hexaedro trilineal Hex8 (ocho nodos), primer orden.*
- *[Pendiente] Hexaedro serendípito Hex20 (veinte nodos), segundo orden.*
- *[Pendiente] Hexaedro lagrangiano Hex27 (veintisiete nodos), segundo orden.*

4.2.6 Familias adicionales no implementadas

- *[Pendiente] Elementos de cáscara y lámina (formulación de Mindlin-Reissner para cáscaras gruesas; formulación de Kirchhoff-Love para cáscaras delgadas).*
- *[Pendiente] Elementos mixtos desplazamiento-presión (Q1-P0, Taylor-Hood) para problemas casi incompresibles.*
- *[Pendiente] Elementos de interfaz para modelado de fricción y contacto cohesivo entre superficies.*

4.2.7 Contrato común y convención de signos

Todos los elementos tipo armadura, cable o marco implementan el contrato `internal_forces(U)` y devuelven los esfuerzos en la convención de signos del proyecto ([ADR 0002](#)).

4.3 Solvers no lineales

El solver no lineal corresponde al nivel estratégico del cálculo: orquesta la subdivisión en pasos, las iteraciones internas y los criterios de convergencia. Es el componente que el usuario selecciona en el archivo YAML mediante el campo `solver.type`.

Disponibles en la versión actual:

- **LinearSolver** — un único paso, una única resolución del sistema $K \cdot U = F$. Aplicable a problemas estrictamente lineales (rigidez constante, sin actualización de geometría, sin no linealidad material).
- **NonlinearSolver** — método de Newton-Raphson incremental con control de carga. Aplica fracciones crecientes de la carga total y, en cada paso, itera hasta la convergencia. Criterio de parada dual (desplazamiento y fuerza).
- **ArcLengthSolver** — método de Crisfield. Introduce una incógnita adicional (el factor de carga) y una restricción geométrica sobre la trayectoria en el espacio (U, λ) , lo que permite recorrer ramas con derivada infinita o negativa. Imprescindible para problemas con *snap-back* o *snap-through*.
- **ModalSolver** ([ADR 0009](#) fase 1) — análisis modal: problema generalizado de valores y vectores característicos $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. No es análisis dinámico propiamente dicho (no hay paso de tiempo ni evolución temporal); sus soluciones admiten interpretación como frecuencias propias y formas de vibración libre no amortiguada. Internamente delega en **EigenSolver** (ARPACK Lanczos con shift-invert) sobre el sistema reducido por Dirichlet. Devuelve **ModalResult** con frecuencias, períodos y modos M-ortonormales; expone además `free_vibration(M, u0, u0_dot, t)` para reconstruir analíticamente la respuesta libre por superposición modal.
- **NewmarkSolver** ([ADR 0009](#) fase 3) — análisis dinámico transitorio lineal: integración temporal directa de $M \cdot \ddot{u} + C \cdot \dot{u} + K \cdot u = F(t)$ por el método de Newmark- β con (β, γ) parametrizables (default 1/4, 1/2 — *average acceleration*, incondicionalmente estable, error $O(\Delta t^2)$). Amortiguamiento Rayleigh $C = \alpha \cdot M + \beta \cdot K$ con coeficientes directos o calibrados modalmente a partir de dos pares (ξ, ω) . Cargas dependientes del tiempo por callback Python. Devuelve **TransientResult** con historiales $(t, u, \dot{u}, \ddot{u})$. La matriz efectiva $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$ se factoriza una sola vez al inicio y se reutiliza en todos los pasos.

- **NewtonNewmarkSolver** ([ADR 0009](#) fase 4) — análisis dinámico transitorio no lineal. **Variante** de **NewmarkSolver** (Reglas §4): hereda predictores, correctores y reducción de Dirichlet; añade un bucle Newton-Raphson dentro de cada paso temporal sobre el residuo dinámico $R = F_{\text{ext}} - F_{\text{int}}(u) - C \cdot \dot{u} - M \cdot \ddot{u}$ con jacobiano $J = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K_t$ (rigidez tangente corriente). Convergencia dual ([ADR 0007](#)). Amortiguamiento Rayleigh constante en el tiempo, calibrado con K_0 (rigidez elástica de referencia; convención estándar Abaqus/ANSYS/OpenSees). Recupera el caso lineal a paridad de bits cuando los materiales no tienen historia activa. Newton modificado opcional (`freeze_tangent_after_iter`, [ADR 0003](#) fase 2).
- **HHTSolver** y **NewtonHHTSolver** ([ADR 0009](#), variantes HHT- α) — disipación numérica controlada en altas frecuencias (Hilber-Hughes-Taylor 1977) con parámetro $\alpha \in [-1/3, 0]$ y (β, γ) auto-derivados. Subclases de **NewmarkSolver** y **NewtonNewmarkSolver** respectivamente (Reglas §4 — variantes).
- **CentralDifferenceSolver** ([ADR 0009](#) fase 5) — integración explícita por diferencias centradas (leapfrog Belytschko-Liu-Moran). M^{-1} trivial sobre masa lumped diagonal. Lineal y no lineal en una sola clase con parámetro `nonlinear`. Estabilidad condicional CFL con detección a posteriori de divergencia.
- **HarmonicSolver** ([ADR 0009](#) fase 6) — respuesta forzada armónica en el dominio de la frecuencia. Aritmética compleja con `scipy.sparse.linalg.spsolve` complejo; factorización LU compleja por frecuencia (no se cachea). Barrido lineal/logarítmico/explicito. Devuelve **HarmonicResult** con métodos `.amplitude()` y `.phase()`.
- **ResponseSpectrumSolver** ([ADR 0009](#) fase 7) — análisis sísmico por combinación modal SRSS/CQC contra espectro de respuesta. Orquesta **ModalSolver** interno y delega los algoritmos en `solidum.math.modal_response` (centralización por regla D de auditoría). Devuelve **ResponseSpectrumResult** con respuesta envolvente, factores de participación y masas efectivas.

Espacio natural de extensión:

- *[Pendiente] Newton-Raphson con búsqueda lineal (line-search) para mejorar la robustez ante no convergencia. **Parcialmente entregado por ADR 0011** — line search disponible como opt-in (`line_search=True`); default `False` por la justificación bibliográfica de Borst & Sluys 1999.*
- *[Pendiente] Solver cuasi-Newton tipo BFGS para reducir el coste de actualización de la matriz tangente.*
- *[Pendiente] Generalized- α , Bossak- α — esquemas adicionales de la familia con disipación numérica.*
- *[Pendiente] Solver de relajación dinámica para búsqueda de configuraciones de equilibrio en estructuras flexibles.*
- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, regla 100/30/30) y multi-support seismic excitation.*
- *[Pendiente] Δt adaptativo para transitorios.*

La incorporación de un solver nuevo se realiza en `solidum/math/solvers/<nombre>.py` mediante el decorador `@SolverRegistry.register`. El **dispatch a entypoints** en `solidum.entry.run_yaml` es declarativo (regla C de auditoría aplicada 2026-05-18): cada solver expone un atributo de clase `PIPELINE_KIND` $\in \{\text{"static", "modal", "transient", "harmonic", "spectrum"}\}$ y `run_yaml` despacha por valor, sin tocarse cuando entran solvers nuevos no clásicos.

4.4 Subsistema algebraico

Por debajo del solver no lineal, dentro de cada iteración, se requiere la resolución de un sistema lineal $K \cdot x = b$. Esta es la responsabilidad del subsistema algebraico (`solidum/math/linalg/`, [ADR 0003](#)). Constituye un nivel táctico: no decide la estrategia del cálculo, sino que selecciona el algoritmo numérico adecuado para resolver cada sistema lineal.

Algoritmos disponibles:

- **LUSolver** — factorización LU mediante SuperLU. Algoritmo universal: aplicable a cualquier matriz no singular, simétrica o no, definida o no.
- **CholeskySolver** — factorización de Cholesky mediante CHOLMOD. Significativamente más rápido y con menor consumo de memoria, pero aplicable únicamente cuando K es simétrica y definida positiva.
- **EigenSolver** ([ADR 0009](#)) — algoritmo de autovalor generalizado simétrico para el problema $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. Envuelve `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). Vive en `solidum/math/linalg/eigen.py` y no comparte la interfaz `solve(K, b)` de los anteriores — su firma natural es `solve(K, M, n_modes) → (λ, φ)`.
- *[Pendiente] **LDLTSolver** — factorización LDL^T para el caso simétrico no definido positivo, reservada para la fase 2 del [ADR 0003](#).*
- *[Pendiente] Algoritmos algebraicos directos paralelos (Pardiso, MUMPS) para problemas de gran tamaño.*
- *[Pendiente] Algoritmos algebraicos iterativos con preconditionamiento (gradiente conjugado preconditionado, GMRES, multimalla algebraica).*

Despachador. La función `select_solver(props)` inspecciona las propiedades de la matriz (encapsuladas en `StiffnessProperties`: simetría, definición positiva) y selecciona el algoritmo adecuado de forma automática. El usuario puede forzar un algoritmo específico desde el archivo YAML mediante el campo opcional `linear_algebra` en calidad de herramienta de diagnóstico, no como decisión de modelado.

Justificación de la separación en dos niveles. El solver no lineal y el subsistema algebraico resuelven cuestiones distintas. El primero decide cómo recorrer la respuesta del sistema (incrementos, iteraciones, longitud de arco). El segundo decide con qué método resolver cada sistema lineal individual. Esta separación permite, por ejemplo, mejorar el rendimiento del paso elástico sin afectar al método de Newton-Raphson, o introducir longitud de arco sin reescribir el código del subsistema algebraico.

4.5 Tipos de análisis

El tipo de análisis en Solidum FEM corresponde al problema físico que afronta el solver, no al solver en sí.

Implementados en la versión actual:

- *Estática lineal*. Desplazamientos pequeños, rigidez constante, una resolución directa. Combinación: `LinearSolver` con materiales lineales y elementos en formulación lineal.
- *Estática no lineal incremental*. No linealidad geométrica (corrotacional), material (plasticidad, daño) o ambas. Combinación: `NonlinearSolver` o `ArcLengthSolver` con materiales no lineales y/o elementos corrotacionales.
- *Análisis modal* ([ADR 0009](#) fase 1). Problema algebraico de valores y vectores característicos generalizado $\mathbf{K} \cdot \boldsymbol{\varphi} = \omega^2 \cdot \mathbf{M} \cdot \boldsymbol{\varphi}$. Combinación: `ModalSolver` con materiales que declaren `density` y elementos que implementen `compute_mass_matrix`.
- *Dinámica estructural transitoria lineal* ([ADR 0009](#) fase 3). Integración temporal directa de la ecuación de movimiento por Newmark- β , con amortiguamiento Rayleigh proporcional y cargas dependientes del tiempo por callback Python. Combinación: `NewmarkSolver` con material lineal, elementos en formulación lineal y `density` declarada.
- *Dinámica estructural transitoria no lineal* ([ADR 0009](#) fase 4). Integración Newmark con Newton-Raphson dentro de cada paso sobre el residuo dinámico $\mathbf{R} = \mathbf{F}_{\text{ext}} - \mathbf{F}_{\text{int}}(\mathbf{u}) - \mathbf{C}\dot{\mathbf{u}} - \mathbf{M}\ddot{\mathbf{u}}$. Combinación: `NewtonNewmarkSolver` con cualquier material con historia (`Elastoplastic1D`, `VonMises2D` en ambas hipótesis, `DruckerPrager2D`, `IsotropicDamage1D/2D`) y `density` declarada. Habilita sísmica con disipación plástica, impacto en hormigón friccional, fatiga de bajo ciclo y vibraciones de marcos elastoplásticos.
- *Dinámica estructural transitoria con disipación HHT- α* ([ADR 0009](#), variante de fases 3 y 4). Mismas combinaciones que Newmark, con la familia HHT- α que controla la disipación numérica de modos altos espurios.
- *Dinámica estructural transitoria explícita* ([ADR 0009](#) fase 5). Integración explícita por diferencias centradas. Combinación: `CentralDifferenceSolver` con masa lumped (`lumping="lumped"`) y la condición de estabilidad $\text{CFL } \Delta t < L_{\text{el}} \cdot \sqrt{(\rho/E)}$ respetada por el usuario. Apropiado para *wave propagation*, impacto y dinámica de respuesta rápida.
- *Respuesta forzada armónica en el dominio de la frecuencia* ([ADR 0009](#) fase 6). Resolución directa del sistema complejo $(-\omega^2 \mathbf{M} + i\omega \mathbf{C} + \mathbf{K}) \cdot \hat{\mathbf{u}} = \mathbf{F}$ por barrido en ω . Combinación: `HarmonicSolver` con amortiguamiento Rayleigh y excitación armónica.
- *Análisis sísmico por combinación modal espectral* ([ADR 0009](#) fase 7). Cálculo modal interno + factores de participación + combinación SRSS o CQC contra un espectro de respuesta tabulado o callable. Combinación: `ResponseSpectrumSolver` con materiales lineales, `density` declarada y un espectro normativo o equivalente.

Previstos y no implementados (dirección del proyecto, sin compromiso de calendario):

- *[Pendiente] Problema térmico estacionario lineal y no lineal (conductividad dependiente de la temperatura).*

- *[Pendiente] Problema térmico transitorio con esquemas de integración temporal (Crank-Nicolson, theta-método).*
- *[Pendiente] Acoplamiento termo-mecánico desacoplado (staggered) para casos en que la influencia mecánica sobre el campo térmico sea despreciable.*
- *[Pendiente] Acoplamiento termo-mecánico monolítico para problemas con fuerte interacción bidireccional.*
- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, 100/30/30) y multi-support para diferencias de movimiento entre apoyos.*

La estructura de capas y registros se ha diseñado para incorporar dichos análisis sin reformular las piezas existentes; el ADR correspondiente se redactará al inicio de cada fase de diseño.

Capítulo 5

Mecanismos transversales

Los patrones que se describen a continuación estructuran el código de Solidum FEM sin pertenecer a una capa concreta. Aparecen indistintamente en materiales, elementos y solvers; su comprensión es necesaria para la lectura de cualquier zona del programa.

5.1 Registros con auto-registro

Cada categoría de componente dispone de su propio diccionario global: `MaterialRegistry`, `ElementRegistry`, `SolverRegistry`. Cada clase concreta se inscribe en su diccionario mediante un decorador (`@MaterialRegistry.register` y análogos) que se ejecuta en el momento de definición de la clase. El intérprete del archivo YAML no requiere conocimiento *a priori* de los materiales o elementos existentes: consulta el registro por el nombre que aparece en el archivo. La incorporación de un componente nuevo no exige la modificación de listas centrales.

5.2 Descubrimiento automático

El módulo `solidum/autodiscover.py` se invoca una sola vez durante la importación del paquete. Recorre, mediante `pkgutil.iter_modules`, las carpetas canónicas (`solidum/materials`, `solidum/elements`, `solidum/math`) e importa cada módulo. Como los decoradores `@register` se ejecutan en el momento de la importación, esta operación basta para poblar todos los registros sin enumeración manual. Este mecanismo es el equivalente conceptual al `INCLUDE` automático de Fortran moderno, ejecutado a tiempo de ejecución.

5.3 Contratos declarativos mediante atributos de clase

En lugar de imponer métodos abstractos cuando el comportamiento puede inferirse del dato, Solidum FEM declara los contratos como atributos de clase: `STRAIN_DIM` (1, 3 o 6), `DOF_NAMES` (lista de nombres de DOF por nodo), `N_INTEGRATION_POINTS`, `PRIMARY_STATE_VAR`. La clase base los lee y se autoconfigura: registra los DOF, valida la compatibilidad material↔elemento e inicializa el `ElementState` con la forma correcta. Este mecanismo sustituye los métodos `setup()` repetitivos que cada subclase tendría que implementar.

5.4 Validación temprana en construcción

Durante la instanciación de un elemento, su método `__init__` verifica que el material recibido sea dimensionalmente compatible. Un elemento `Truss2D` con material `Elastic2D` produce un error en la construcción del caso, no después de doscientas iteraciones en forma de fallo opaco durante la solución. El coste es una comprobación trivial; el beneficio consiste en que ciertos errores físicos se detectan de forma inmediata en el momento de su introducción.

5.5 Intérprete genérico por introspección de argumentos

El componente `YamlParser` no contiene ramificaciones del tipo `if material_type == .Elastic1D` por cada material. Inspecciona los argumentos del constructor de la clase recuperada del registro y le transmite los campos del archivo YAML. La incorporación de un material nuevo no requiere modificación del intérprete; es suficiente con que el constructor declare los parámetros con nombres compatibles con el archivo YAML.

5.6 Estados *trial* y comprometido en `ElementState`

Cada elemento mantiene un objeto `ElementState` con dos copias de las variables internas: una copia *trial* (la que se explora durante las iteraciones del solver) y una copia comprometida (la que corresponde al último paso convergido). El solver invoca `commit_state()` solo cuando un paso converge, lo que evita la contaminación del historial plástico o de daño con tentativas posteriormente descartadas. Esta semántica es crítica para problemas con plasticidad o daño: en su ausencia, una iteración no convergida que casualmente quedase dentro de tolerancia contaminaría el historial de forma irreversible.

5.7 Ensamblaje disperso con caché en formato `Coordinate`

El primer ensamblaje de la matriz global calcula los pares de índices (i, j) de cada contribución elemental y los almacena en formato `Coordinate` (COO). En las iteraciones siguientes, se reescribe únicamente el vector de datos sobre la misma topología, sin recálculo de índices. Se trata del patrón “calcular una vez, reutilizar” típico en programas de MEF compilados, implementado aquí sobre `scipy.sparse`.

5.8 Eliminación directa de condiciones de frontera

Las condiciones de frontera, tanto Dirichlet nodales como restricciones multipunto lineales (MPC), se imponen por eliminación directa ([ADR 0004](#)). Toda restricción se expresa en forma afín $\mathbf{u}_s = \mathbf{g}_s + \sum \alpha_{si} \cdot \mathbf{u}_{mi}$ y se acumula en un `ConstraintSet` (subpaquete `solidum.bc`). El ensamblador construye un operador disperso $\mathbf{T}$ y un vector $\mathbf{g}$ tales que $\mathbf{u} = \mathbf{T} \cdot \mathbf{u}_{libre} + \mathbf{g}$; en filas correspondientes a DOF libres, $\mathbf{T}$ es la matriz identidad rectangular; en filas esclavas, lleva los coeficientes α_{si} en las columnas de los maestros. El sistema entregado al subsistema algebraico es $\mathbf{K}_{red} = \mathbf{T}^T \mathbf{K} \mathbf{T}$, $\mathbf{F}_{red} = \mathbf{T}^T (\mathbf{F} - \mathbf{K} \cdot \mathbf{g})$. La imposición es exacta a redondeo, preserva la simetría y la positividad definida de $\mathbf{K}$, y deja el sistema reducido apto para solvers iterativos (CG, GMRES).

La forma afín cubre con la misma maquinaria los casos de empotramiento, asentamiento prescrito, apoyo en plano oblicuo, periodicidad de celda unitaria y unión rígida master-slave entre nodos.

Las restricciones declaradas como cadenas (un esclavo cuyo maestro es a su vez esclavo de otra restricción) se resuelven por cierre transitivo en el momento de construir `T`; los ciclos y las redeclaraciones inconsistentes se detectan en validación temprana. La declaración se hace mediante `Domain.add_linear_constraint` o desde el bloque `linear_constraints` del archivo YAML.

5.9 Compilación Just-In-Time mediante Numba

Las funciones críticas (ensamblaje elemento→global, núcleo del algoritmo de retorno radial) se decoran con `@jit` y se compilan a código nativo en su primera invocación. La primera ejecución asume el coste de compilación; las invocaciones siguientes se ejecutan a velocidad cercana a la de un programa Fortran compilado. La restricción consiste en que el código compilado solo admite tipos primitivos y arreglos NumPy, no objetos arbitrarios de Python.

5.10 Variable principal de visualización

Cada material declara como atributo de clase la variable interna principal a efectos de visualización: por ejemplo `'damage'` en un modelo de daño o `'alpha'` en plasticidad con endurecimiento. El componente `VtkExporter` la lee de forma genérica sin conocimiento del material que la origina. Este mecanismo permite la incorporación de materiales nuevos con visualización automática, sin modificación del exportador.

5.11 Cuadraturas centralizadas

Las tablas y reglas de cuadratura de Gauss-Legendre 1D, 2D y 3D residen centralizadas en `solidum/math/integration.py`. Cada elemento declara su `N_INTEGRATION_POINTS` y consume los puntos y pesos correspondientes, sin reproducción de tablas en cada subclase.

5.12 Política unificada de tolerancias: patrón `atol + rtol · escala`

Toda comparación con significado físico o iterativo del proyecto — admisibilidad constitutiva, convergencia de Newton-Raphson, convergencia de arc-length, futuros criterios de cierre de gap en contacto o de inversión del jacobiano — sigue una misma fórmula estructural:

```
magnitud      atol + rtol · escala(estado)
```

donde `atol` es un piso absoluto en las unidades físicas del problema, `rtol` es una banda relativa adimensional y `escala(estado)` es la magnitud característica del criterio en el estado corriente. Esta forma única garantiza tres propiedades simultáneamente: invariancia bajo cambio de unidades (la `escala` se ajusta), adaptatividad al estado (la tolerancia crece con la evolución del problema, p. ej. con el endurecimiento plástico) y robustez en regímenes degenerados (cuando la escala colapsa transitoriamente, `atol` mantiene la comparación significativa).

El patrón vive centralizado por subsistema, no replicado: la admisibilidad constitutiva se aplica en `Material.is_admissible` y `Material.admissibility_tol` ([ADR 0006](#)); la convergencia de los solvers no lineales reside en la clase `ConvergenceCriterion` de `solidum/math/convergence.py` ([ADR 0007](#)). Cada material o solver declara su `escala` mediante un método ligero (por ejemplo `Material.admissibility_scale`, que devuelve la fluencia corriente o el umbral de daño), y la fórmula no se reproduce a mano en ningún sitio salvo cuando una restricción técnica (kernel

compilado con Numba) obliga a precomputar la tolerancia fuera del kernel — y, aun así, vía el método centralizado.

Adicionalmente, los términos absolutos `atol` se autoderivan de las escalas del problema en su primer ensamblaje, no se codifican como constantes globales con unidades. Las constantes globales del proyecto que rigen esta política son adimensionales (`CONVERGENCE_RTOL_FORCE`, `CONVERGENCE_ATOL_FORCE_FACTOR`, `ADMISSIBILITY_TOL_REL`, etc.), lo que mantiene el código independiente del sistema de unidades elegido por el usuario.

La importancia de este mecanismo es estructural: la convergencia es lo que marca el éxito de un análisis numérico, y la diferencia entre un código robusto y uno frágil está en la disciplina con que se construyen estas comparaciones. Toda extensión futura que introduzca un nuevo criterio de comparación contra cero adopta este patrón.

Capítulo 6

Convenciones de signos

La convención de signos es única para el conjunto del proyecto y de aplicación obligatoria en toda formulación, prueba, interfaz pública (`internal_forces`, `SolveResult`), documentación y catálogo. Si una referencia bibliográfica adopta otra convención, la traducción se realiza al implementar y se anota en la prueba correspondiente.

6.1 Ejes y giros

Estática 2D. Eje x positivo a la derecha; eje y positivo hacia arriba. Giro y momento positivos en sentido antihorario (regla de la mano derecha (RHR) con z saliente del plano).

Estática 3D. Aplicación de la RHR a todos los ejes (locales y globales) y a todos los momentos (M_x , M_y , M_z , T).

6.2 Esfuerzos internos en 2D — convención de viga estructural

Sobre un elemento diferencial, con N positivo en tracción, V positivo cuando tiende a rotar el diferencial en sentido horario y M positivo en flexión sagitada (tracción en la fibra inferior con $+y$ hacia arriba):

- *Cara izquierda* (normal saliente en $-x$): N apunta en $-x$, V apunta en $+y$, M actúa en sentido horario.
- *Cara derecha* (normal saliente en $+x$): N apunta en $+x$, V apunta en $-y$, M actúa en sentido antihorario.

Esta es la convención clásica de los diagramas de esfuerzos en vigas.

6.3 Esfuerzos internos en 3D — convención de resultantes de tensión bajo RHR

En la cara con normal saliente $+x_{\text{local}}$, los esfuerzos positivos se definen como:

- N en $+x_{\text{local}}$ (tracción positiva).
- V_y en $+y_{\text{local}}$, V_z en $+z_{\text{local}}$.

- $T \quad M_x, M_y, M_z$: vectores de momento en $+x_{\text{local}}, +y_{\text{local}}, +z_{\text{local}}$ respectivamente, conforme a la RHR.

En la cara con normal saliente $-x_{\text{local}}$, todos los sentidos se invierten (tercera ley de Newton).

Justificación: esta es la convención que resulta de integrar directamente el tensor de tensiones sobre la sección. Es la adoptada por Bathe, Crisfield, Cook-Malkus-Plesha, SAP2000, OpenSees, ANSYS y Abaqus para vigas 3D. La convención estructural de "flexión sagitada positiva" no admite extensión canónica a 3D y se utiliza únicamente en 2D.

Convención de fibra para los flectores en 3D, consecuencia directa del signo:

- $M_z > 0 \Rightarrow$ tracción en fibras con $y < 0$ (equivalente a flexión sagitada en el plano xy con $+y$ hacia arriba).
- $M_y > 0 \Rightarrow$ tracción en fibras con $z > 0$.

6.4 Capa interna y capa pública

Internamente, todos los elementos (2D y 3D) operan en la convención de resultantes de tensión bajo RHR. Las matrices B , las fuerzas internas $f_{\text{int}} = \int B^T \sigma \, d\Omega$, los jacobianos y los residuos se calculan en dicha convención. Esta uniformidad evita signos especiales por dimensión dentro del código de formulación.

En la interfaz pública (`internal_forces`, diagramas, catálogo), los elementos 3D la exponen sin transformación. Los elementos 2D, en cambio, exponen la convención de viga estructural (V con signo opuesto al interno) por ser la representación clásica en los diagramas de esfuerzos. La traducción consiste en un simple cambio de signo en V , aplicado dentro del método `internal_forces()` del elemento 2D.

Relación 2D $\leftrightarrow$ 3D en la interfaz pública:

- $M_{2D} \quad M_{z_{3D}}$ (mismo signo; ambos corresponden a flexión sagitada positiva con $+y$ hacia arriba).
- V_{2D} y $V_{y_{3D}}$ difieren en signo en la interfaz pública por la razón anterior.

Los valores se exponen siempre en ejes locales del elemento; la transformación a ejes globales constituye una capa separada.

Capítulo 7

Crecimiento del sistema

Los capítulos anteriores describen el estado actual del programa. Este capítulo describe el procedimiento por el que se incorporan piezas nuevas — el aspecto del proyecto que la arquitectura optimiza explícitamente.

7.1 Tres puntos de extensión canónicos

Componente a incorporar	Ubicación	Mecanismo de registro	Modifica otros archivos
Material	<code>solidum/materials/<snake></code>	<code>@MaterialRegistry.register</code>	No
Elemento	<code>solidum/elements/<snake></code>	<code>@ElementRegistry.register</code>	No
Solver	<code>solidum/math/solver_<snake></code>	<code>@SolverRegistry.register</code>	No

La columna "modifica otros archivos" ^{es} deliberadamente "No". Si la incorporación de un componente exigiera la modificación del intérprete, el ensamblador o la inicialización, dicha situación constituiría un síntoma de degradación de la arquitectura y procedería un refactor con su correspondiente Architecture Decision Record (ADR).

7.2 Skill `/solidum-new`

Para reducir también la fricción ergonómica de la incorporación, el repositorio incluye una *skill* versionada (`.claude/skills/solidum-new/`) que el asistente invoca cuando el usuario solicita un material, elemento o solver nuevo. La invocación es `/solidum-new material|element|solver <Name>` y produce automáticamente:

- El archivo en su carpeta canónica con la plantilla del tipo correspondiente.
- El decorador `@register` correctamente posicionado.
- Una prueba esqueleto en `tests/` con la estructura habitual (configuración, expectativas, validación contra solución analítica si aplica).

La *skill* no sustituye al razonamiento físico ni a la formulación; constituye estrictamente un mecanismo de generación de plantillas. Cierra el ciclo entre la arquitectura (optimizada para extensión) y la herramienta operativa que la materializa.

7.3 Ciclo especificación → código → catálogo

Cuando el componente a incorporar es físico (no infraestructura), el flujo es:

1. *Apertura de la especificación.* El usuario abre una especificación en `docs/specs/<Nombre>.md` a partir de la plantilla `_template_<kind>.md`. La especificación contiene: especificación física, formulación numérica y contrato YAML (`interface`, `parameters`, `acceptance`, `references`).
2. *Bloqueo en ausencia de especificación completa.* El asistente no escribe código hasta que la especificación esté completa. Si falta información, formula consultas en la sección *Diálogo* de la propia especificación. Esto convierte la especificación, sucesivamente, en orden de trabajo y en referencia detallada del componente.
3. *Implementación.* El asistente implementa el componente y añade pruebas conforme a los criterios de `acceptance`.
4. *Cierre.* Tras la validación (todas las pruebas verdes contra los criterios), el asistente actualiza `status: validated` en la especificación e inserta una entrada en el catálogo correspondiente (`docs/catalogo_<elementos|materiales|solvers>.md`).

La relación es: la especificación constituye la orden de trabajo y la referencia detallada por componente; el catálogo constituye el índice navegable del conjunto. El manual de referencia (`Reference_manual.pdf`) se genera de forma automática a partir de las especificaciones.

7.4 Architecture Decision Records: memoria de arquitectura persistente

Para los cambios que afectan al diseño del programa (no a la implementación interna de un módulo), el asistente crea un Architecture Decision Record en `docs/adr/000N-titulo.md` antes del `commit` o de forma simultánea al mismo. Los ADR son persistentes y consultables; sustituyen a las explicaciones efímeras del *chat* a fin de que el sistema permanezca comprensible meses o años después. Los ADR vigentes se enumeran en el siguiente capítulo.

7.5 Mantenimiento de los documentos vivos

Los archivos `docs/arquitectura.md`, `docs/conceptos_clave.md` y este manual se mantienen sincronizados con el estado del código. El asistente los actualiza cuando se modifica la topología del sistema (introducción de una capa nueva, una carpeta canónica nueva o una responsabilidad transversal nueva), no cuando se modifica la implementación interna de un módulo. Si tras un refactor de tamaño medio cualquiera de estos documentos no refleja la realidad del código, dicha discrepancia constituye un defecto del propio refactor.

Capítulo 8

Decisiones de arquitectura (ADR)

Los Architecture Decision Records (ADR) son el registro persistente de las decisiones que han modelado la arquitectura de Solidum FEM. Cada ADR responde, en una página, a tres cuestiones: contexto, decisión y consecuencias. En este capítulo se resumen los ADR vigentes; el texto completo reside en `docs/adr/`.

8.1 ADR 0001 — Mecanismos de transparencia conceptual

Fecha: 19 de abril de 2026. **Estado:** aceptado.

Contexto. El documento `Reglas.md` §2 establece el modelo “calculadora con manual disponible”: el usuario delega la escritura del código pero conserva su comprensión conceptual. Para que dicho principio se opere, y no permanezca como mera declaración de intenciones, el proyecto requiere mecanismos prácticos que prevengan la deriva hacia la opacidad a medida que el código crece.

Decisión. Se introducen tres mecanismos vivos, de mantenimiento económico y mutuamente reforzantes:

1. *Mapa de arquitectura.* El archivo `docs/arquitectura.md` consiste en una página con diagrama de bloques y flujo de datos. El asistente lo actualiza al modificarse la topología del sistema.
2. *Conceptos clave.* El archivo `docs/conceptos_clave.md` contiene un inventario explícito de los conceptos sostenedores del sistema, cada uno con dos o tres frases conceptuales. Define el “test de no opacidad”: el usuario debe poder explicar cualquier entrada en términos generales en cualquier momento.
3. *Modo .*explícame X*”bajo demanda.* Protocolo conversacional, no artefacto en repositorio. Constituye la válvula de escape ante opacidad puntual sin necesidad de sesiones programadas.

Consecuencia para este manual. El presente `Architecture_manual.pdf` es el siguiente eslabón en la cadena de transparencia conceptual: amplía el mapa de arquitectura a una lectura cómoda en formato de libro, manteniendo la disciplina de no entrar al detalle de cada componente.

8.2 ADR 0002 — Interfaz pública de resultados para consumidores externos

Fecha: 22 de abril de 2026. **Estado:** aceptado.

Contexto. Solidum FEM se consume desde una GUI externa (FenixBAR, programa de análisis estructural con elementos barra). Tras una solución, el consumidor requiere información suficiente para visualizar el post-proceso estándar de Computer-Aided Engineering (CAE): deformada escalada, reacciones en apoyos y diagramas de esfuerzos internos sobre cada barra. Esta información no se exponía de forma uniforme: el componente `VtkExporter` cubría únicamente algunos elementos y los esfuerzos internos carecían de método público homogéneo.

Decisión. Interfaz pública estructurada en tres niveles:

1. *Contrato por elemento.* Todos los elementos tipo armadura, cable o marco implementan `internal_forces(U) ->ElementForces`, que devuelve los esfuerzos en ejes locales con claves homogéneas por familia (`{N}` para armadura y cable; `{N, V, M}` para marco 2D; `{N, Vy, Vz, T, My, Mz}` para marco 3D). La convención de signos es la del proyecto (capítulo anterior).
2. *Agregado en Domain.* El objeto `SolveResult` se calcula de forma anticipada al final de `solver.solve()` y es inmutable. Expone los desplazamientos globales, las cargas aplicadas, las reacciones y un diccionario de esfuerzos internos por elemento.
3. *Puntos de entrada oficiales.* Las funciones `solidum.run(case)` y `solidum.run_yaml(path)` devuelven directamente un `SolveResult`. Cualquier consumidor externo opera contra esta interfaz.

Consecuencia. El consumidor externo no recalcula esfuerzos a partir de `U`; la lógica de MEF reside íntegramente en Solidum FEM. La incorporación de un elemento nuevo obliga a implementar `internal_forces` con las claves de su familia: el contrato es explícito.

8.3 ADR 0003 — Despachador de algoritmo algebraico

Fecha: 26 de abril de 2026. **Estado:** aceptado; fases 1 y 2 implementadas.

Contexto. Dentro de cada iteración del método de Newton-Raphson se requiere la resolución del sistema lineal $K \cdot \delta U = R$. El algoritmo utilizado hasta el momento era SuperLU (factorización LU general), válido para todo caso pero subóptimo cuando K es simétrica y definida positiva, situación habitual en estática lineal y en numerosas no linealidades suaves. La factorización de Cholesky es típicamente entre 1.5 y 2 veces más rápida y consume considerablemente menos memoria en dichos casos.

Decisión. Se introduce un subsistema algebraico independiente del solver no lineal, con varios algoritmos disponibles y un despachador que selecciona el adecuado de forma automática:

- `LUSolver` (SuperLU) — algoritmo universal.
- `CholeskySolver` (CHOLMOD) — para matrices simétricas y definidas positivas; opcional según disponibilidad de `scikit-sparse`.
- `LDLTSolver` — reservado para la fase 2 (matrices simétricas indefinidas).

El despachador `select_solver(props, override)` recibe un objeto `StiffnessProperties` (atributos `is_symmetric`, `is_positive_definite`) y selecciona la factorización de Cholesky cuando aplica, o la factorización LU en caso contrario. El usuario puede forzar un algoritmo desde el archivo YAML mediante el campo `linear_algebra` en calidad de herramienta de diagnóstico, no como decisión de modelado.

Consecuencia. Los solvers no lineales (`LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`) ya no invocan SuperLU directamente: solicitan un solver algebraico al despachador y se desentienden del algoritmo numérico subyacente. La incorporación de un nuevo algoritmo (Pardiso, MUMPS, métodos algebraicos multimalla iterativos) afecta exclusivamente a `solidum/math/linalg/`. La separación entre estrategia (solver no lineal) y táctica (subsistema algebraico) queda explícita en el código.

8.4 ADR 0004 — Imposición de condiciones de Dirichlet por eliminación directa

Fecha: 4 de mayo de 2026. **Estado:** aceptado; fases 1 (Dirichlet nodal) y 2 (restricciones lineales afines) implementadas.

Contexto. Las condiciones de contorno de Dirichlet, tanto los empotramientos nodales clásicos como las restricciones multipunto (MPC: asentamientos prescritos, apoyos en plano oblicuo, periodicidad de celda unitaria, unión rígida master-slave), se imponían previamente por método de penalidad: añadir términos diagonales muy grandes al sistema. Dicho método introduce condicionamiento numérico artificialmente malo, contamina la simetría y la positividad de la matriz de rigidez y no es exacto a redondeo.

Decisión. Se adopta la imposición por eliminación directa. Toda restricción se expresa en forma afín $u_s = g_s + \sum \alpha_{si} \cdot u_{mi}$ y se acumula en un `ConstraintSet` del subpaquete `solidum.bc`. El ensamblador construye un operador disperso T y un vector g tales que $u = T \cdot u_{libre} + g$. El sistema entregado al subsistema algebraico es $K_{red} = T^T K T$, $F_{red} = T^T (F - K \cdot g)$. Las restricciones encadenadas (un esclavo cuyo maestro es a su vez esclavo) se resuelven por cierre transitivo al construir T ; los ciclos y redeclaraciones inconsistentes se detectan en validación temprana.

Consecuencia. La imposición es exacta a redondeo, preserva simetría y positividad definida de K , y deja el sistema reducido apto para los algoritmos del subsistema algebraico, incluidos solvers iterativos como gradiente conjugado o GMRES. La misma maquinaria cubre empotramiento, asentamiento, apoyo oblicuo, periodicidad y unión rígida sin ramificación específica por tipo.

8.5 ADR 0005 — Logging configurable en lugar de print directo

Fecha: 7 de mayo de 2026. **Estado:** aceptado.

Contexto. La trazabilidad del solver durante un análisis (incrementos de carga, iteraciones de Newton, ratios de convergencia, factorizaciones reusadas) se emitía mediante `print` directo en la salida estándar. Esto impedía silenciar el ruido en pipelines de batch testing, redirigir la traza a un archivo de log paralelo y diferenciar niveles de gravedad (información ordinaria, advertencias, errores recuperables).

Decisión. Se introduce un *logger* configurable basado en el módulo estándar `logging` de Python, expuesto al resto del proyecto vía `solidum.logging.get_logger(name)`. Todos los componentes consumen este *logger* en lugar de `print`. El nivel y el formato se configuran de forma centralizada y pueden sobrescribirse desde el archivo YAML del caso o programáticamente.

Consecuencia. Tests automatizados pueden silenciar la traza sin perder la información de errores. La integración con herramientas externas (GUI FenixBAR, batería de tests, pipelines CI) se hace mediante la API estándar de `logging`, sin acoplamiento a `sys.stdout`. Las advertencias internas relevantes (degradación de Cholesky a LU, bisección de incremento) quedan marcadas con su nivel adecuado.

8.6 ADR 0006 — Tolerancias del criterio de admisibilidad constitutiva

Fecha: 12 de mayo de 2026. **Estado:** aceptado.

Contexto. Cada modelo constitutivo no lineal tiene un criterio de admisibilidad — una función $f(\text{estado})$ que decide si el material está en régimen reversible ($f \leq 0$) o disipativo ($f > 0$). En aritmética de doble precisión la comparación no puede ser exacta: tras un return mapping, el ruido de redondeo deja f cerca de cero pero no exactamente cero, por lo que la comparación se escribe $f \leq \text{tol}$. La implementación anterior usaba una tolerancia absoluta hardcodeada (`PLASTIC_YIELD_TOL = 1e-9`), frágil bajo cambio de unidades y no adaptativa al estado en modelos con endurecimiento severo o sin parámetro de referencia inicial.

Decisión. Se adopta el esquema combinado absoluto + relativo, estilo solver de ecuaciones diferenciales ordinarias: $f \leq \text{ADMISSIBILITY_TOL_ABS} + \text{ADMISSIBILITY_TOL_REL} \cdot \text{escala}(\text{estado})$. Cada material declara su escala característica vía el método `admissibility_scale(state_vars)`, devolviendo un esfuerzo positivo característico del criterio en el estado corriente: $\sigma_y + H \cdot \alpha$ para plasticidad J2 1D, $E \cdot \kappa_0$ para daño isótropo, $\sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)}$ para Von Mises 2D. La comparación se encapsula en `Material.is_admissible(f, state_vars)` y en `Material.admissibility_tol(state_vars)` para que los modelos no la repliquen.

Consecuencia. Misma física en MPa, Pa, GPa o adimensional produce idéntico estado interno hasta precisión de doble. En endurecimiento severo la tolerancia crece junto con la fluencia. Modelos energéticos futuros (phase-field, gradient damage) encajan declarando su escala ($\sqrt{(2 \cdot E \cdot g_c)}$) sin tocar la política. Los modelos con kernel compilado (Numba) consumen `admissibility_tol` desde código Python antes de invocar el kernel, preservando la centralización de la fórmula.

8.7 ADR 0007 — Tolerancias de convergencia en solvers no lineales

Fecha: 13 de mayo de 2026. **Estado:** aceptado.

Contexto. El criterio de convergencia del `NonlinearSolver` y del `ArcLengthSolver` era puramente relativo: $\|\text{residuo}\| / \text{ref_force} \leq \text{tol}$ y $\|\delta U\| / \|U\| \leq \text{tol}$, con un único parámetro `tol` para los dos criterios y sin término absoluto real. Esta forma arrastraba tres limitaciones estructurales: no había tolerancia absoluta cuando la escala relativa colapsaba transitoriamente, una sola tolerancia mezclaba dos criterios físicamente distintos (incrementabilidad del iterado vs equilibrio), y la fórmula se replicaba en los dos solvers a mano.

Decisión. Se aplica el mismo patrón del [ADR 0006](#) a la convergencia, separado por criterio. La política reside en una clase pequeña `ConvergenceCriterion` en `solidum/math/convergence.py`, que encapsula configuración (cuatro tolerancias: `rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`) y estado calibrado (los `atol` efectivos derivados de las escalas del primer ensamblaje). Los solvers no lineales reciben una instancia en el constructor (parámetro `convergence`), calibran al inicio de `solve()` y delegan en `evaluate()` la comparación. La semántica es AND, no OR (ambos criterios deben cumplirse simultáneamente), coherente con la convención de Bathe, Crisfield y Owen-Hinton.

Consecuencia. Invariancia bit-paritaria bajo cambio de unidades (el mismo problema en N/m, kN/mm o MN/m converge en el mismo número de iteraciones). Régimen degenerado (carga inicial nula, control en desplazamiento puro) deja de oscilar espuriamente: el piso absoluto `atol_force` autoderivado define cuándo se da por convergido un residuo numéricamente cero. Problemas casi-rígidos y plásticos perfectos se afinan independientemente vía `rtol_disp` vs `rtol_force` sin compromisos. La política vive en un único punto del código; un solver no lineal futuro la consume sin replicarla.

8.8 ADR 0008 — Densidad como propiedad del material

Fecha: 13 de mayo de 2026. **Estado:** aceptado.

Contexto. El cableado de fuerza de cuerpo introducido previamente aceptaba un vector `body_force`: `[bx, by, bz]` global, aplicable uniformemente a todos los elementos. Esto servía para estructuras monomaterial — el usuario calculaba $\rho \cdot g$ una vez y lo declaraba — pero fallaba en estructuras multimaterial: con un único vector global, no había forma de distinguir el peso propio del acero del hormigón o de un cable tensado. Adicionalmente, la densidad es la magnitud física que abre la puerta a la matriz de masa elemental y por tanto a todo el subsistema dinámico futuro.

Decisión. La densidad es propiedad del material, no del elemento. Se introduce como atributo `density` en la clase base `Material` con valor por defecto `None` (no declarado). La densidad es físicamente intrínseca al medio continuo; vive en el mismo nivel jerárquico que `E`, ν o σ_y . En el YAML se acepta el bloque atajo `gravity`: `[0, -9.81]` complementario a `body_force` (mutuamente excluyentes). El ensamblador incorpora `assemble_self_weight(g)` que itera sobre elementos y aplica `b_element = material.density · g`. Si algún material tiene `density=None`, el método falla con `ValueError` listando los materiales afectados — enforcement diferida al uso, sin posibilidad de propagar masa cero silenciosa a un análisis dinámico.

Consecuencia. Peso propio físicamente correcto en estructuras multimaterial. API del YAML más intuitiva. Tests existentes con `body_force` siguen pasando. La fase de análisis modal y dinámico hereda la propiedad sin rediseño: `Assembler.assemble_mass_matrix` reutiliza el mismo patrón de validación.

8.9 ADR 0009 — Análisis modal y dinámico

Fecha: 13 de mayo de 2026. **Estado:** aceptado; **todas las fases (1-7) implementadas** al 18 de mayo de 2026, junto con la variante HHT- α y las reglas C y D de refactor arquitectural aplicadas.

Contexto. El [ADR 0003](#) había identificado análisis modal y dinámica implícita entre los análisis futuros que la capa algebraica iba a habilitar; el [ADR 0008](#) había puesto la densidad como propiedad del material disponible. Quedaba abrir el subsistema con dos análisis distintos pero relacionados: el análisis modal — problema algebraico de valores y vectores característicos generalizado $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ —, cuyas soluciones admiten interpretación dinámica pero no constituyen análisis dinámico per se; y el análisis dinámico propiamente dicho, con dependencia temporal explícita — integración de $M \cdot \ddot{u} + C \cdot \dot{u} + K \cdot u = F(t)$.

Decisión. Se acometen ambos análisis con una hoja de ruta común y siete decisiones arquitecturales fijadas hoy para no reabrir debates en fases posteriores:

1. Matriz de masa **consistente** en la fase 1 (con contrato extensible a `lumped` mediante un parámetro `lumping` ya presente en la firma de `compute_mass_matrix`, suficiente para la futura integración explícita).
2. Ensamblaje de `M` paralelo al de `K`, reutilizando la topología COO cacheada del `Assembler` y la enforcement de densidad del [ADR 0008](#).
3. `ModalSolver` registrado en el mismo `SolverRegistry` que los solvers estáticos: no se crea un `IntegratorRegistry` paralelo, manteniendo el patrón “un solver = un análisis = una clase registrada”.
4. Dirichlet en el problema modal por eliminación directa (mismo mecanismo del [ADR 0004](#)); el término independiente `g` de restricciones lineales no aplica en modal.

5. Estado dinámico transitorio ($\mathbf{u}$, $\dot{\mathbf{u}}$, $\ddot{\mathbf{u}}$) fuera de `Node`: vive en un `TransientResult` específico, preservando la pureza topológica/geométrica de la clase nodal y evitando que análisis estáticos paguen el coste de campos que no usan.
6. Amortiguamiento Rayleigh $\mathbf{C} = \alpha \cdot \mathbf{M} + \beta \cdot \mathbf{K}$ como entrada estándar para la fase de dinámica transitoria, con coeficientes calibrables modalmente a partir de dos pares (ξ, ω) . Amortiguamiento modal por modo y otros esquemas (Caughey, local) diferidos.
7. Convención de unidades heredada del modelo ([ADR 0008](#)): el usuario es responsable de la consistencia; las tolerancias adimensionales del [ADR 0007](#) garantizan invariancia bajo cambio de unidades.

Las siete fases quedan implementadas y validadas con tests contra solución analítica y recuperación a paridad de bits del caso lineal en ausencia de plasticidad:

- **Fase 1** (`ModalSolver` + ARPACK Lanczos con shift-invert). Cerrada 2026-05-13.
- **Fase 2** (mass lumping HRZ canónico en sólidos + nodal directo en frames). Helper centralizado `solidum.math.mass_lumping.lump_hrz`. Cerrada 2026-05-18 (commit [c92bf4e](#)).
- **Fase 3** (`NewmarkSolver` con (β, γ) parametrizables, factorización única, Rayleigh, cargas por callback). Cerrada 2026-05-13.
- **Fase 3-bis** (`HHTSolver` como variante de Newmark con disipación numérica controlada por $\alpha \in [-1/3, 0]$). Cerrada 2026-05-18 (commit [73742f4](#)).
- **Fase 4** (`NewtonNewmarkSolver` con Newton interno, jacobiano $\mathbf{J} = \mathbf{M} + \gamma \Delta t \cdot \mathbf{C} + \beta \Delta t^2 \cdot \mathbf{K}_t$, convergencia dual [ADR 0007](#), Rayleigh constante con `K_0`; `NewtonHHTSolver` análogo). Cerrada 2026-05-13/-18.
- **Fase 5** (`CentralDifferenceSolver` — leapfrog explícito Belytschko-Liu-Moran, lineal y no lineal en una clase con `nonlinear`, estabilidad condicional CFL con detección a posteriori). Cerrada 2026-05-18 (commit [e66473d](#)).
- **Fase 6** (`HarmonicSolver` — respuesta forzada armónica $(-\omega^2 \mathbf{M} + i\omega \mathbf{C} + \mathbf{K}) \cdot \hat{\mathbf{u}} = \mathbf{F}$ con barrido configurable, aritmética compleja). Cerrada 2026-05-18 (commit [47a1333](#)).
- **Fase 7** (`ResponseSpectrumSolver` — combinación modal SRSS/CQC sobre espectro de respuesta). Cerrada 2026-05-18 (commit [5151d7d](#)).

Reglas arquitecturales aplicadas durante el cierre del ADR:

- **Regla C** (auditoría 2026-05-13). Disparada por `CentralDifferenceSolver` como primer solver fuera de la jerarquía de Newmark. `entry.py::run_yaml` despacha por atributo de clase `PIPELINE_KIND` $\in \{\text{"static", "modal", "transient", "harmonic", "spectrum"}\}$ en lugar de cadena de `isinstance`. Solvers no clásicos futuros no requieren tocar `entry.py`.
- **Regla D** (auditoría 2026-05-13). Disparada por `ResponseSpectrumSolver` como segundo método de cómputo sobre `ModalResult`. El algoritmo de `free_vibration` migra a `solidum/math/modal_response` junto con los nuevos `participation_factors`, `response_spectrum_srss`, `response_spectrum_cqc`, `spectrum_from_sa`, `spectrum_tabulated`. `ModalResult.free_vibration` queda como wrapper delgado, preservando la API histórica.

Consecuencia. El subsistema modal/dinámico/espectral queda **completo en su totalidad**. Las únicas extensiones futuras (excitación sísmica multi-direccional simultánea CQC3, multi-support seismic, Δt adaptativo, generalized- α) requieren caso de uso específico — no son piezas del ADR pendientes. La clase `Node` conserva su semántica original.

8.10 Evolución de esta lista

Cada decisión de arquitectura de gran calado — refactor transversal, subsistema nuevo, ruptura de contratos — produce un ADR adicional. Los siguientes ADR se prevén en las fases de diseño futuras:

- *[Pendiente] ADR para la introducción del problema térmico, incluyendo la generalización del concepto de DOF a magnitudes escalares no mecánicas.*
- *[Pendiente] ADR para el acoplamiento termo-mecánico, incluyendo la elección entre estrategia desacoplada y monolítica.*

Capítulo 9

Evolución de la arquitectura

Este capítulo presenta la trayectoria prevista de la arquitectura de Solidum FEM. Su función es preservar la imagen de la arquitectura del programa a medida que crece: cada fase incorpora un nuevo tipo de análisis con sus contratos propios, sus variables de estado y, eventualmente, capas o subsistemas adicionales. La diferencia con los pendientes del [capítulo 4](#) es de granularidad: aquellos enumeran componentes individuales; este capítulo describe los **estados sucesivos de la arquitectura** del programa, cada uno con su mapa de capas correspondiente.

Las fases posteriores a la actual están marcadas como pendientes y se materializarán mediante los Architecture Decision Records (ADR) anunciados en el [capítulo 8](#). La estructura de cada fase se redefinirá en su ADR de apertura; lo aquí descrito constituye una hipótesis razonada de partida.

9.1 Fase 1 (actual) — Estática lineal y no lineal

Estado: implementado.

Capas activas: las seis capas descritas en el [capítulo 2](#) (entrada, inicialización, interpretación del caso, dominio, capa numérica, salida).

Tipos de análisis: estática lineal (`LinearSolver`) y estática no lineal incremental (`NonlinearSolver`, `ArcLengthSolver`).

Variables de estado: desplazamientos `U`. Variables internas del material (deformaciones plásticas, daño, historia) gestionadas mediante `ElementState` con semántica *trial* / comprometido.

Matrices globales: matriz de rigidez `K` (única).

Contrato de elemento: `K_local`, `f_internal`, `internal_forces(U)`. La matriz `B` y el residuo se calculan en cada iteración.

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, solvers] - [Salida]
```

9.2 Fase 2 — Análisis modal

Estado: implementado ([ADR 0009](#) fase 1, sesión del 2026-05-13).

El análisis modal resuelve el problema generalizado de valores y vectores característicos $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. Es un problema algebraico cuyas soluciones admiten **interpretación** como frecuencias propias y formas de vibración libre no amortiguada, pero el análisis en sí es estático: no hay paso de tiempo ni evolución temporal. Se acometió antes que la dinámica transitoria porque solo requiere la matriz de masa nueva, aísla esa pieza, y deja la infraestructura preparada para Newmark.

Cambios respecto a la fase 1:

- Aparece la matriz de masa M , ensamblada con la misma infraestructura que K (mismo patrón de sparsity, misma topología COO cacheada).
- Cada elemento implementa `compute_mass_matrix(lumping)` — masa consistente translacional invariante en barras y cables, axial + Hermitiana cúbica + inercia rotacional propia de sección en frames 2D/3D, integral $\int \rho \cdot \mathbf{N}^T \mathbf{N} \cdot \mathbf{t} \, dA$ en sólidos 2D (Tri6 usa cuadratura específica `tri_6` por subintegración con `tri_3`).
- Nuevo subsistema en la capa algebraica: `EigenSolver` que envuelve ARPACK Lanczos con shift-invert (`scipy.sparse.linalg.eigsh`).
- Nueva clase de solver: `ModalSolver`, registrado en `SolverRegistry` (no se fragmenta el registro). Reutiliza Dirichlet por eliminación directa con un `reduce_pair(K, M)` que aplica T a ambas matrices simultáneamente.
- Nueva salida especializada: `ModalResult` (frozen) con `frequencies_rad/hz`, `periods`, `modes` M -ortonormales, y un método `free_vibration(M, u0, u0_dot, t)` que reconstruye analíticamente la respuesta libre sin amortiguamiento por superposición modal — sin necesidad de integrador temporal.
- `Material.density` ([ADR 0008](#)) ya disponía de la única propiedad física nueva necesaria.

Topología efectiva:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, M, eigensolver]
          - [Salida: ModalResult]
```

Riesgo de arquitectura

Bajo. El análisis modal añade un nuevo tipo de problema algebraico (autovalor generalizado en lugar de sistema lineal) y una nueva familia de resultados, sin modificar contratos existentes ni la infraestructura de Dirichlet.

9.3 Fase 3 — Dinámica estructural transitoria lineal

Estado: implementado ([ADR 0009](#) fase 3, sesión del 2026-05-13).

Integración temporal directa de la ecuación de movimiento semidiscreta $M \cdot \ddot{\mathbf{u}} + C \cdot \dot{\mathbf{u}} + K \cdot \mathbf{u} = \mathbf{F}(t)$ por el método de Newmark- β . Es el primer análisis con dependencia temporal real; aprovecha la matriz de masa introducida en la fase 2 y reutiliza las decisiones arquitecturales fijadas en el [ADR 0009](#) (registro unificado, estado dinámico fuera de `Node`, amortiguamiento Rayleigh como entrada estándar).

Cambios respecto a la fase 2:

- Aparece la matriz de amortiguamiento $C = \alpha \cdot M + \beta \cdot K$ (Rayleigh proporcional). Los coeficientes pueden pasarse directos o calibrarse modalmente a partir de (ξ_1, ω_1) , (ξ_2, ω_2) con `solidum.math.damping.rayleigh_from_modes`.
- Las variables de estado se amplían a $(\mathbf{u}, \dot{\mathbf{u}}, \ddot{\mathbf{u}})$. Coherente con el [ADR 0009](#) §5, el estado dinámico **no contamina** la clase `Node`: vive en el nuevo `TransientResult` (frozen) como historiales `t_history`, `u_history`, `udot_history`, `uddot_history` indexados por paso temporal.

- Nuevo solver `NewmarkSolver` con esquema (β, γ) parametrizable (default 1/4, 1/2 — average acceleration, incondicionalmente estable, error $\mathcal{O}(\Delta t^2)$). Cargas dependientes del tiempo por callback Python (`F_func: t → F`).
- El subsistema algebraico recibe una matriz efectiva $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$. Con Δt constante y problema lineal, A_{eff} es invariante: se factoriza **una sola vez** al inicio ([ADR 0003](#) — `FactorizedSolver`) y cada paso temporal se reduce a una sustitución triangular barata.
- Aceleración inicial calculada de forma consistente con la ecuación de movimiento: $M \cdot \ddot{u}_0 = F(0) - C \cdot \dot{u}_0 - K \cdot u_0$.

Topología efectiva:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, M, C, A_eff,
  solver] - [Salida: TransientResult]
                                     |
                                     [Integrador temporal
                                     Newmark]
```

Riesgo de arquitectura

Bajo. La decisión 5 del [ADR 0009](#) (estado dinámico fuera de `Node`, en `TransientResult`) preserva la pureza topológica/geométrica de `Node` y evita que análisis estáticos paguen el coste de campos que no usan. La semántica *trial* / comprometido del estado interno de elementos (ADR sobre `ElementState`) sigue intacta porque la fase 3 es lineal: no hay variables internas que evolucionen.

Limitaciones declaradas en esta fase 3:

- Solo lineal: K , M constantes. No-linealidad (Newton dentro de Newmark) entregada en la fase 4.
- Apoyos Dirichlet constantes en el tiempo. Multi-support excitation sísmica diferida.
- Δt fijo; adaptativo diferido.
- Esquemas con amortiguamiento numérico controlado (HHT- α) entregados en 2026-05-18 como variante de la familia Newmark.

9.4 Fase 4 — Dinámica estructural transitoria no lineal

Estado: implementado ([ADR 0009](#) fase 4, sesión del 2026-05-14).

Cambios respecto a la fase 3:

- Nueva clase `NewtonNewmarkSolver` como **variante** de `NewmarkSolver` (Reglas §4 — extensión documentada en spec corta, sin ADR nuevo porque reusa todas las decisiones del [ADR 0009](#)): hereda predictores, correctores y reducción de Dirichlet; añade un bucle Newton-Raphson dentro de cada paso temporal.
- En cada paso, el residuo $R = F_{\text{ext}}(t_{\{n+1\}}) - F_{\text{int}}(u_{\{n+1\}}) - C \cdot \dot{u}_{\{n+1\}} - M \cdot \ddot{u}_{\{n+1\}}$ se anula iterativamente con jacobiano $J = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K_t$, donde $K_t = \partial F_{\text{int}} / \partial u$ es la rigidez tangente corriente. Convergencia dual del [ADR 0007](#) calibrada al primer paso.

- Reaprovecha íntegramente la maquinaria del `NonlinearSolver` estático: `ConvergenceCriterion`, `freeze_tangent_after_iter` (Newton modificado, [ADR 0003](#) fase 2), `commit_all_states()` tras converger cada paso temporal — el estado interno de los elementos recupera plenamente su semántica *trial* / comprometido también en dinámica.
- Amortiguamiento Rayleigh $C = \alpha \cdot M + \beta \cdot K_0$ **constante en el tiempo**, calibrado con la rigidez elástica de referencia K_0 al inicio del análisis. Convención estándar (Abaqus, ANSYS, OpenSees); evita acoplamiento ad-hoc entre disipación viscosa y plástica.
- **Recuperación del caso lineal:** con materiales sin historia (o plásticos no plastificados), $K_t = K_0$ y el residuo se anula en una iteración. El resultado coincide a paridad de bits con `NewmarkSolver`, validado en tests.
- Despacho YAML automático: como `NewtonNewmarkSolver` es subclase de `NewmarkSolver`, `entry.run_yaml` lo detecta vía `isinstance` y enruta a `run_transient` sin tocar el dispatcher.

Topología efectiva: igual que la fase 3, con el bucle Newton añadido dentro del bucle temporal y la rigidez tangente reensamblada en cada iteración.

Riesgo de arquitectura

Bajo en la práctica: toda la infraestructura ya existía. El `NonlinearSolver` estático aportaba calibración de convergencia, fallback de positividad y Newton modificado; la fase 3 aportaba predictores/correctores Newmark y reducción de Dirichlet. La fase 4 es la composición exacta de ambas, formalizada como subclase para no duplicar código.

9.5 Fase 4 (extensión 2026-05-18) — Cierre completo del subsistema modal/dinámico/espectral

Estado: implementado ([ADR 0009](#) fases 2, 5, 6, 7 + variante HHT- α + reglas C y D de auditoría aplicadas).

Las cuatro fases pendientes del [ADR 0009](#) se cierran en una secuencia obligada (cada una precondition de la siguiente) sin reabrir las decisiones arquitecturales tomadas en el ADR original:

Mass lumping (fase 2) — `compute_mass_matrix(lumping="lumped")` operativo en todos los elementos. Helper centralizado `solidum.math.mass_lumping.lump_hrz` con esquema HRZ canónico (Hinton-Rock-Zienkiewicz 1976) para sólidos isoparamétricos y fórmula nodal directa para frames. Diagonalidad estricta en globales para todos los elementos excepto `Frame3D` con eje oblicuo (queda bloque-diagonal — limitación documentada, Cook-Malkus-Plesha §11.4).

HHT- α (variante) — `HHTSolver` y `NewtonHHTSolver` como variantes de la familia Newmark (Reglas §4 — spec corta sin ADR nuevo). Disipación numérica controlada por $\alpha \in [-1/3, 0]$ con (β, γ) auto-derivados canónicamente.

Central differences (fase 5) — `CentralDifferenceSolver` con leapfrog explícito Belytschko-Liu-Moran. Una sola clase cubre lineal y no lineal con parámetro `nonlinear`. Requiere masa lumped (`lumping=consistent` rechazado con `ValueError`). Estabilidad condicional CFL con detección a posteriori de divergencia exponencial.

Harmonic (fase 6) — `HarmonicSolver` resuelve $(-\omega^2 M + i\omega C + K) \cdot \hat{u} = F$ con aritmética compleja para un barrido en ω . Factorización LU compleja por frecuencia. Nuevo `HarmonicResult` con métodos `.amplitude()` y `.phase()`. Cuarto valor del literal `PIPELINE_KIND` (ver regla C abajo).

Response spectrum (fase 7) — `ResponseSpectrumSolver` combina respuestas máximas modales bajo un espectro $S_d(\omega)$ o $S_a(\omega)$ por SRSS (default) o CQC (Der Kiureghian 1980, para modos cercanos). Devuelve `ResponseSpectrumResult` con respuesta envolvente, contribución modal individual, factores de participación, masas efectivas y método `.cumulative_effective_mass_ratio()` para verificación normativa (≥ 0.9).

Reglas C y D aplicadas durante esta extensión (auditoría arquitectural 2026-05-13):

- **Regla C** — `entry.py::run_yaml` despacha por atributo de clase `PIPELINE_KIND` $\in \{"static", "modal", "transient", "harmonic", "spectrum"\}$ en lugar de cadena de `isinstance`. Solvers no clásicos futuros no requieren tocar `entry.py`.
- **Regla D** — `solidum/math/modal_response.py` agrupa todos los cálculos sobre modos: `free_vibration` (movido desde `results.py`), `participation_factors`, `response_spectrum_srss`, `response_spectrum_cqc`, helpers de espectros (`spectrum_from_sa`, `spectrum_tabulated`). `ModalResult.free_vibration` queda como wrapper delgado preservando la API histórica. `results.py` vuelve a su propósito declarado: dataclasses inmutables, no algoritmos.

Topología efectiva tras la extensión:

```
[Entrada] - [Inicializacion] - [Interprete: PIPELINE_KIND] - [Dominio] - [Numerica: K,
M, C, lump_hrz, modal_response] - [Salida]
```

```
[run, run_modal, run_transient, run_harmonic,
run_response_spectrum] → [Result type específico]
```

Riesgo de arquitectura

Bajo. Las cinco piezas son extensiones aditivas: nuevo módulo `mass_lumping.py`, nuevo módulo `modal_response.py`, cinco solvers nuevos en `solidum/math/solvers/`, dos `Result` nuevos (`HarmonicResult`, `ResponseSpectrumResult`). El despacho declarativo `PIPELINE_KIND` reemplaza la cadena de `isinstance` sin romper contratos — los solvers existentes heredan el nuevo atributo. La regla D centraliza algoritmos sin modificar la API pública de `ModalResult`.

9.6 Fase 5 — Problema térmico estacionario y transitorio

Estado: *[Pendiente] Implementación de la fase 5 — problema térmico.*

Cambios respecto a las fases anteriores:

- Aparece un campo escalar nuevo: la temperatura `T`. Esto obliga a generalizar el concepto de DOF: hasta ahora todos los DOF eran mecánicos (desplazamientos, rotaciones); ahora se incorporan DOF térmicos.
- Aparece la matriz de conductividad `K_t` (análogo térmico de la rigidez) y la matriz de capacidad `C_t` (análogo térmico de la masa).
- Se introduce una nueva familia de elementos térmicos, paralela a la mecánica. Cada elemento térmico declara `DOF_NAMES = ['T']` y consume materiales con propiedades térmicas (conductividad, capacidad, fuente).
- Aparece una nueva familia de materiales térmicos con interfaz propia: `compute_flux_and_tangent(grad_T, state)`.

- En transitorio, se reutilizan los integradores temporales de la fase 2, adaptados al sistema térmico (Crank-Nicolson, theta-método).

Topología prevista:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K_m, M_m, K_t, C_t
, solvers] - [Salida]
                                     |
                               [DOF mecanicos] + [DOF termicos]
                               [Materiales mec.] + [Materiales term.]
                               [Elementos mec.] + [Elementos term.]
```

Riesgo de arquitectura

Medio. Es la primera fase que rompe la suposición implícita "todos los DOF son mecánicos" presente hoy en numerosos puntos del código. La generalización requiere atributo `DOF_KIND` o similar en cada DOF (mecánico, térmico, futuro presión, futuro químico); posible separación entre `MechanicalElement` y `ThermalElement` con base común `Element`, o un único `Element` polimórfico (decisión a tomar en el ADR de apertura); adaptación del `VtkExporter` para escribir campos escalares además de vectoriales.

9.7 Fase 6 — Acoplamiento termo-mecánico

Estado: *[Pendiente] Implementación de la fase 6 — acoplamiento termo-mecánico.*

Cambios respecto a la fase 5:

- Aparece la interacción entre los campos térmico y mecánico: dilatación térmica que induce deformaciones, disipación mecánica que genera calor.
- Dos estrategias distintas en términos de arquitectura, a decidir en el ADR de apertura:
- *Estrategia desacoplada (staggered):* en cada paso, se resuelve primero el problema térmico con los desplazamientos de la última iteración, después el problema mecánico con las temperaturas recién calculadas. Cada subsistema mantiene su propia matriz; el acoplamiento se realiza mediante intercambio de campos. Implementación de bajo riesgo de arquitectura pero convergencia limitada en problemas de fuerte interacción.
- *Estrategia monolítica:* se ensambla un sistema único `K_acoplada` que combina los DOF mecánicos y térmicos. Convergencia robusta pero exige una nueva infraestructura de ensamblaje multifísico.
- Aparece una nueva familia de materiales acoplados con interfaz extendida: `compute_response(strain, T, grad_T, state)`.

Topología prevista (estrategia monolítica):

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K_acoplada,
M_acoplada, solvers] - [Salida]
                                     |
                               [Materiales termo-mec.]
                               [Elementos termo-mec.]
                               [Solver acoplado]
```

Riesgo de arquitectura

Alto. Es la primera fase con auténtica multifísica. Requiere generalización del subsistema algebraico para manejar matrices con bloques no homogéneos (un bloque mecánico simétrico positivo definido y un bloque térmico no simétrico, en la formulación habitual); generalización del concepto de prueba de validación, ya que no basta una solución analítica monodisciplinaria y los problemas de referencia deben tener acoplamiento conocido; posible introducción de preconditionadores específicos para sistemas acoplados (precondicionamiento por bloques, complemento de Schur).

9.8 Mantenimiento de este capítulo

Cada vez que se cierre la implementación de una fase y se acepte su ADR de apertura, esta sección se actualiza:

1. La fase pasa de "[*Pendiente*]^a implementado".
2. La hipótesis inicial sobre la arquitectura se sustituye por la descripción real, ajustada a las decisiones que se hayan tomado durante la implementación.
3. El diagrama de capas se redibuja conforme a la topología efectiva.
4. Las consecuencias no previstas (ajustes en capas anteriores, contratos modificados) se registran en una subsección "Lecciones de la fase N", de cara a fases posteriores.

De este modo, el capítulo se convierte en una crónica ordenada del crecimiento del programa: en cualquier momento futuro, abrir el manual permite reconstruir no solo el estado actual de Solidum FEM sino la trayectoria completa que lo ha llevado hasta él.

Capítulo 10

Compromisos de diseño

Los Architecture Decision Records (ADR) del [capítulo 8](#) registran las decisiones de arquitectura que han sido **tomadas e implementadas**. Este capítulo registra el complemento simétrico: las decisiones que han sido **deliberadamente postergadas o rechazadas**, junto con la justificación de su no adopción. La función es preservar memoria sobre los caminos no recorridos, de modo que cuestiones ya resueltas no se reabran innecesariamente en sesiones futuras y que, si en algún momento las circunstancias del proyecto cambian, la revisión parta de un punto informado y no desde cero.

A diferencia de los ADR, las entradas de este capítulo no se materializan en archivos individuales en `docs/adr/`: viven exclusivamente aquí. Cada entrada se actualiza cuando las circunstancias que motivaron la decisión cambien sustancialmente.

10.1 Paralelización con MPI

Decisión: postergada sin fecha.

Contexto. La paralelización mediante el estándar Message Passing Interface (MPI) sería el paso natural para llevar Solidum FEM a problemas de gran escala (mallas de millones de elementos, descomposición de dominio, ejecución en clúster).

Razones de la postergación. Un solo desarrollador trabaja en el proyecto y los problemas que motivan su uso (investigación en mecánica de sólidos, validación de modelos constitutivos, prototipado de algoritmos) se ejecutan satisfactoriamente en máquinas de un solo nodo. La introducción de MPI multiplicaría el coste de cualquier extensión por la necesidad de mantener consistencia de estado entre procesos, complicaría sustancialmente las pruebas (ejecución determinística reproducible bajo distintas particiones de malla) y exigiría rediseñar el ensamblaje y el subsistema algebraico. El compromiso entre coste de mantenimiento y beneficio inmediato no es favorable mientras el caso de uso del proyecto se mantenga en su perfil actual.

Reapertura de la decisión

Procedería revisar este compromiso si Solidum FEM se aplicara de forma habitual a mallas mayores que 10 elementos, o si se incorporasen colaboradores adicionales con experiencia en programación paralela.

10.2 Adopción de PETSc o Trilinos como backend numérico

Decisión: rechazada para el alcance actual.

Contexto. PETSc y Trilinos son las dos bibliotecas estándar para álgebra lineal dispersa, solvers iterativos, descomposición de dominio y gestión de matrices distribuidas en programas de elementos finitos de gran escala. Su adopción daría acceso inmediato a multimalla algebraica, preconditionadores avanzados y soporte MPI maduro.

Razones del rechazo. Ambas son dependencias binarias pesadas con un proceso de instalación no trivial en Windows, plataforma de trabajo del desarrollador. Su modelo de programación es de matriz distribuida y orientación procedural, lo que choca con la arquitectura actual basada en `scipy.sparse` y orientación a objetos. La capacidad de Solidum FEM hoy se ajusta a lo que SuperLU y CHOLMOD ofrecen mediante `scipy.sparse` y `scikit-sparse`; añadir PETSc o Trilinos sin un caso de uso concreto que lo exija supondría una dependencia infrutilizada y un coste de mantenimiento desproporcionado.

Reapertura de la decisión

Procedería si se adopta MPI (compromiso anterior) y, simultáneamente, los problemas tratados requirieran solvers iterativos con preconditionamiento avanzado. En ese escenario, PETSc se convertiría en la opción natural por encima de implementar dichos algoritmos en código propio.

10.3 Arquitectura cliente-servidor o servicio web

Decisión: rechazada.

Contexto. Una arquitectura cliente-servidor expondría Solidum FEM como un servicio remoto al que la GUI externa (FenixBAR) y otros consumidores se conectarían vía protocolo HTTP, gRPC o similar. Es el modelo dominante en simulación industrial moderna como servicio.

Razones del rechazo. El consumidor actual (FenixBAR) es local, está bajo control del mismo desarrollador y no tiene requisitos de concurrencia ni de despliegue distribuido. La interfaz pública se ha estabilizado mediante el [ADR 0002](#) con `SolveResult`, `solidum.run` y `solidum.run_yaml` en proceso, lo que cubre el caso de uso de consumidor externo sin la sobrecarga de un protocolo de red. La introducción de un servidor exigiría serialización de mallas y resultados, gestión de sesiones, autenticación si el servicio fuera multiusuario, y un ciclo de despliegue paralelo al del programa. Ninguna de estas necesidades existe hoy.

Reapertura de la decisión

Procedería si Solidum FEM se ofreciera como servicio público a múltiples consumidores no relacionados con el desarrollo, o si la GUI se desacoplara hasta el punto de ejecutarse en máquinas distintas.

10.4 Soporte de múltiples lenguajes en la interfaz pública

Decisión: rechazada para el alcance actual.

Contexto. La interfaz pública del programa (`solidum.run`, `solidum.run_yaml`, `SolveResult`) es accesible exclusivamente desde Python. Una alternativa sería exponer la misma interfaz en C, C++ o como biblioteca compartida llamable desde Fortran y MATLAB, lo que ampliaría el espectro de consumidores potenciales.

Razones del rechazo. El consumidor identificado (FenixBAR) está escrito en Python; el ecosistema de programas docentes y de investigación al que se dirige Solidum FEM utiliza Python

como lenguaje dominante. Ofrecer un binding multilingüe duplicaría el esfuerzo de mantenimiento de la interfaz pública sin un caso de uso concreto que lo justifique. La integración con MATLAB, si fuera necesaria en algún momento, se realizaría mediante el mecanismo nativo de MATLAB para llamar a Python (`py.solidum.run_yaml(path)`), sin necesidad de modificar Solidum FEM.

Reapertura de la decisión

Procedería si surgiera un consumidor estable escrito en un lenguaje distinto de Python y la solución *vía* el puente nativo de cada lenguaje no fuera viable.

10.5 Sistema de plugins externo al repositorio

Decisión: rechazada.

Contexto. Algunos programas grandes de elementos finitos permiten que terceros publiquen materiales, elementos o solvers como paquetes externos que se descubren en tiempo de ejecución sin formar parte del repositorio principal. La alternativa es la actual: todo componente vive dentro del repositorio de Solidum FEM y se valida con sus pruebas.

Razones del rechazo. La filosofía del proyecto (ver [capítulo 1](#)) considera la transparencia y la auditabilidad del código como valor primordial. Un sistema de plugins externos abriría la puerta a componentes no auditados que podrían silenciosamente romper la convención de signos del proyecto, los contratos declarativos del catálogo o las hipótesis de validación contra soluciones analíticas. Mientras el desarrollo lo lleve un único equipo, la incorporación de cualquier componente nuevo pasa por el repositorio y por las pruebas correspondientes, lo que mantiene la trazabilidad completa.

Reapertura de la decisión

Procedería si el proyecto creciera hasta tener una comunidad amplia de contribuyentes externos con necesidades especializadas no absorbibles en el catálogo principal. Hoy no es el caso ni se prevé que lo sea.

10.6 Refactor a tipado estático estricto con `mypy --strict`

Decisión: postergada sin fecha.

Contexto. El código de Solidum FEM utiliza anotaciones de tipo en sus interfaces públicas pero no se valida con `mypy` en modo estricto. Adoptarlo proporcionaría garantías estáticas adicionales y detectaría algunas familias de errores antes de la ejecución.

Razones de la postergación. La librería `numpy` y, en menor medida, `scipy.sparse` tienen un soporte de tipado estático que ha mejorado mucho pero sigue dejando puntos donde `mypy --strict` produce ruido sin valor real. El esfuerzo de pulir todas esas fricciones no se compensa, en el alcance actual del proyecto, con el beneficio en detección de defectos. La política presente — anotaciones de tipo en interfaces públicas y validación de comportamiento mediante pruebas de física — se considera suficiente.

Reapertura de la decisión

Procedería cuando el ecosistema NumPy/SciPy cierre las lagunas de tipado restantes y la activación del modo estricto deje de generar fricción artificial.

10.7 Mantenimiento de este capítulo

Cada vez que se reabra una decisión y se modifique su estado (de "postergada.^a rechazada", o viceversa), o cuando una decisión postergada se materialice en un ADR formal, la entrada correspondiente se actualiza. Si la decisión pasa a aceptarse, su entrada se retira de este capítulo y se sustituye por el ADR correspondiente en el [capítulo 8](#).

Capítulo 11

Glosario

Términos técnicos centrales de Solidum FEM, ordenados alfabéticamente. Cada entrada da una definición breve (de una a tres frases) y, cuando procede, remite al capítulo donde el concepto se desarrolla.

Ajuste de paso (incremento de carga). División de la carga total en fracciones que se aplican sucesivamente en problemas no lineales. Permite que el método de Newton-Raphson converja a partir de una aproximación cercana a la solución del paso anterior.

Ardura corrotacional (formulación). Descomposición del movimiento de un elemento en una rotación de cuerpo rígido más una deformación local pequeña. La ley constitutiva trabaja en la configuración local; la cinemática del elemento absorbe las grandes rotaciones. Aplicable a armaduras, cables y marcos en régimen geoméricamente no lineal con deformación pequeña.

Architecture Decision Record (ADR). Documento breve que registra una decisión de arquitectura junto con su contexto, alternativas consideradas y consecuencias. Reside en `docs/adr/000N-titulo.md`. Ver [capítulo 8](#).

Autodiscover (descubrimiento automático). Procedimiento que recorre las carpetas canónicas del proyecto e importa todos sus módulos al inicio del programa, lo que dispara la ejecución de los decoradores `@register` y, con ello, la inscripción de cada componente en su registro. Ver [capítulo 5](#).

Capa algebraica. Subsistema en `solidum/math/linalg/` responsable de resolver el sistema lineal $K \cdot x = b$ que aparece dentro de cada iteración del solver no lineal. Está separada del solver no lineal y selecciona automáticamente el algoritmo de factorización adecuado mediante un despachador. Ver [capítulo 4](#) y [ADR 0003](#).

Catálogo. Documento navegable que enumera todos los componentes implementados de una categoría (elementos, materiales, solvers) con sus parámetros, validez y referencias. Reside en `docs/catalogo_<categoría>.md`. Es índice y resumen; el detalle vive en las especificaciones.

Contrato declarativo. Mecanismo por el cual una clase declara, mediante atributos de clase como `STRAIN_DIM`, `DOF_NAMES`, `N_INTEGRATION_POINTS` o `PRIMARY_STATE_VAR`, las propiedades fijas que el sistema necesita conocer de ella. La clase base lee estos atributos y se autoconfigura, sustituyendo métodos de inicialización repetitivos en cada subclase. Ver [capítulo 5](#).

Despachador algebraico. Función `select_solver(props, override)` que recibe las propiedades de la matriz de rigidez (simetría, definición positiva) y devuelve el algoritmo de factorización adecuado: Cholesky cuando aplica, LU en caso contrario. Ver [ADR 0003](#).

DOF_NAMES. Atributo de clase de cada elemento que enumera los nombres de los grados de libertad activos en cada uno de sus nodos. La clase base lo lee para registrar los grados de libertad globales en el `Domain`.

Domain. Objeto central de la capa de dominio que mantiene la lista de nodos, la conectividad de los elementos, la numeración global de los grados de libertad y, tras la solución, el agregado `SolveResult`.

Eliminación directa (de condiciones de Dirichlet). Estrategia de imposición de condiciones esenciales que retira los grados de libertad prescritos del sistema antes de resolverlo, en oposición a la penalización (que añade un valor grande a la diagonal) y a los multiplicadores de Lagrange (que amplían el sistema). Toda restricción se expresa en forma afín $\mathbf{u}_s = \mathbf{g}_s + \sum \alpha_{si} \cdot \mathbf{u}_{mi}$ y se acumula en un `ConstraintSet` (`solidum/bc/`). El ensamblador construye un operador disperso $\mathbf{T}$ y un vector $\mathbf{g}$ tales que $\mathbf{u} = \mathbf{T} \cdot \mathbf{u}_{libre} + \mathbf{g}$, y entrega al solver el sistema reducido $\mathbf{K}_{red} = \mathbf{T}^T \mathbf{K} \mathbf{T}$, $\mathbf{F}_{red} = \mathbf{T}^T (\mathbf{F} - \mathbf{K} \cdot \mathbf{g})$. La imposición es exacta a redondeo y preserva la simetría y la positividad definida de la matriz original. Ver [ADR 0004](#).

Elemento. Entidad geométrica que construye su matriz de gradientes $\mathbf{B}$, su matriz de rigidez tangente y su vector de fuerzas internas a partir de los desplazamientos nodales y del material asignado. Sus contratos declarativos están en `solidum/elements/`. Ver [capítulo 4](#).

ElementState. Objeto que encapsula las dos copias de las variables internas de cada elemento: una copia *trial* (la que se explora durante las iteraciones del solver) y otra comprometida (la del último paso convergido). Ver [capítulo 5](#).

Ensamblaje con caché Coordinate (COO). Estrategia de ensamblaje en la que la primera invocación calcula los pares de índices (i, j) de cada contribución elemental y los almacena en formato Coordinate; las invocaciones siguientes reescriben únicamente el vector de datos sobre la misma topología, sin recalculer índices. Ver [capítulo 5](#).

Especificación (spec). Documento que describe un componente físico nuevo antes de su implementación: especificación física, formulación numérica, contrato YAML y criterios de aceptación. Reside en `docs/specs/<Nombre>.md`. Es a la vez orden de trabajo y referencia detallada del componente. Ver [capítulo 7](#).

****Estado *trial* y estado comprometido**.** Las dos copias de las variables internas de cada elemento. El estado *trial* se explora durante las iteraciones del solver no lineal; al converger un paso, el solver invoca `commit_state()` y promueve el *trial* a estado comprometido. Esta semántica es indispensable en problemas con plasticidad o daño para no contaminar el historial con tentativas posteriormente descartadas.

Familia de elementos. Conjunto de elementos que comparten la cinemática esencial: armaduras, cables, marcos (vigas), sólidos 2D, sólidos 3D, cáscaras. Ver [capítulo 4](#).

Grado de libertad (DOF). Cada incógnita escalar del problema discreto. En estática mecánica, los grados de libertad son los desplazamientos nodales y, en su caso, las rotaciones nodales. La numeración global se establece en el `Domain`.

Longitud de arco (método de Crisfield). Estrategia de solución no lineal que introduce el factor de carga como incógnita adicional y añade una restricción geométrica sobre la trayectoria en el espacio $(\mathbf{U}, \lambda)$. Permite recorrer ramas con derivada infinita o negativa, imprescindible para problemas con *snap-back* o *snap-through*.

Material. Ley constitutiva que, dada una deformación y un estado interno, devuelve la tensión y el módulo tangente consistente. Su contrato declarativo está en `solidum/materials/`. Ver [capítulo 4](#).

Método de Elementos Finitos (MEF). Técnica de discretización para resolver ecuaciones diferenciales parciales mediante la subdivisión del dominio en elementos sobre los que se interpolan los campos incógnita. Solidum FEM trabaja en aproximación de desplazamientos.

Multipoint constraint (MPC). Restricción afín lineal que liga el desplazamiento de un grado de libertad esclavo a una combinación lineal de los desplazamientos de uno o varios grados de libertad maestros, posiblemente con un término independiente. Modelan apoyos en plano oblicuo, periodicidad

de celda unitaria, uniones rígidas entre nodos y simetrías no alineadas con los ejes globales. Se declaran mediante `Domain.add_linear_constraint` o desde el bloque `linear_constraints` del archivo YAML, y se imponen por eliminación directa con la misma maquinaria que las condiciones de Dirichlet. Ver [capítulo 5](#) y [ADR 0004](#).

Notación de Voigt. Representación de un tensor simétrico de segundo orden como un vector. En 2D, los tres componentes son las dos componentes normales y la componente de cortante (Voigt-3); en 3D, los seis componentes son las tres normales y las tres de cortante (Voigt-6).

Numba (compilación Just-In-Time, JIT). Decorador `@njit` que compila a código nativo, en la primera invocación, las funciones a las que se aplica. La primera ejecución asume el coste de compilación; las siguientes corren a velocidad cercana a Fortran. Restringido a tipos primitivos y arreglos NumPy.

PRIMARY_STATE_VAR. Atributo de clase de cada material que declara cuál de sus variables internas es la principal a efectos de visualización. El exportador VTK la lee genéricamente sin conocimiento del material que la origina.

Registro (registry). Diccionario global por categoría (`MaterialRegistry`, `ElementRegistry`, `SolverRegistry`) que mapea el nombre de cada componente con su clase. Cada clase se inscribe automáticamente en su registro mediante un decorador. Ver [capítulo 5](#).

Regla de la mano derecha (RHR). Convención de orientación tridimensional para ejes y vectores momento, adoptada universalmente en Solidum FEM para convenciones de signos de magnitudes vectoriales en 3D. Ver [capítulo 6](#).

Retorno radial (return mapping). Algoritmo predictor-corrector para integrar la ecuación constitutiva de plasticidad J2: se asume un paso elástico (predictor), se evalúa el criterio de fluencia y, si la tensión predictora lo viola, se proyecta de vuelta a la superficie de fluencia (corrector). El módulo tangente consistente se obtiene linealizando el algoritmo discreto.

SolveResult. Agregado inmutable que el `Domain` construye al final de la solución. Contiene los desplazamientos globales, las cargas aplicadas, las reacciones y los esfuerzos internos por elemento. Es la interfaz pública para consumidores externos. Ver [ADR 0002](#).

Solver no lineal. Componente que orquesta la solución de un problema no lineal: subdivisión en pasos, iteraciones internas, criterio de convergencia. En Solidum FEM están implementados `LinearSolver`, `NonlinearSolver` y `ArcLengthSolver`. Es nivel estratégico, distinto del subsistema algebraico. Ver [capítulo 4](#).

STRAIN_DIM. Atributo de clase de cada material y elemento que declara la dimensión del vector de deformación que manejan: 1 (escalar axial), 3 (Voigt-3 bidimensional), 6 (Voigt-6 tridimensional). El sistema lo usa para validar la compatibilidad material↔elemento en el momento de construcción.

Tensión plana / deformación plana. Hipótesis de modelado para problemas bidimensionales. La tensión plana asume que la tensión perpendicular al plano es nula (placas delgadas); la deformación plana asume que la deformación perpendicular es nula (cuerpos extensos en una dimensión, secciones de tuberías largas).

Validación temprana en construcción. Comprobación que la clase base de elemento ejecuta cuando se le asigna un material: verifica que la dimensión de deformación del material coincida con la del elemento. Errores físicos de combinación se detectan en la construcción del caso, no doscientas iteraciones después.

Visualization Toolkit (VTK). Conjunto de formatos de archivo y bibliotecas para representación de mallas y campos discretos en visualizadores de simulación numérica como ParaView. El `VtkExporter` de Solidum FEM produce los archivos correspondientes.

YAML (YAML Ain't Markup Language). Formato de serialización legible por humanos en el que se describen los casos de Solidum FEM (nodos, materiales, elementos, condiciones de contorno, cargas, solver). Es contrato público del programa.

Apéndice A

Pendientes de desarrollo

Este apéndice consolida, en una sola vista, todos los elementos marcados a lo largo del manual como pendientes de desarrollo. Cada entrada indica el capítulo de procedencia y el texto del pendiente. La lista se genera de forma automática a partir de los marcadores [PENDIENTE: ...] insertados en las fuentes; cualquier modificación se realiza sobre el archivo fuente correspondiente, no sobre este apéndice.

Bloques funcionales

- *[Pendiente] Modelo de Mohr-Coulomb 2D con esquinas (multi-superficie) para mayor fidelidad geotécnica respecto a Drucker-Prager.*
- *[Pendiente] Endurecimiento cinemático (Bauschinger) en J2 para plasticidad cíclica y fatiga.*
- *[Pendiente] Hiperelasticidad isótropa (Neo-Hooke, Mooney-Rivlin, Ogden).*
- *[Pendiente] Viscoplasticidad de Perzyna y Duvaut-Lions.*
- *[Pendiente] Hiperelasticidad incompresible con tratamiento mixto desplazamiento-presión.*
- *[Pendiente] Drucker-Prager en tensión plana (la proyección con $\sigma_{zz}=0$ acoplada a flujo dilatante queda fuera del alcance de la versión actual).*
- *[Pendiente] Armadura 2D de segundo orden con linealidad geométrica.*
- *[Pendiente] Armadura 2D de segundo orden con linealidad geométrica y material no lineal.*
- *[Pendiente] Armadura 3D de segundo orden con linealidad geométrica.*
- *[Pendiente] Armadura 3D de segundo orden con linealidad geométrica y material no lineal.*
- *[Pendiente] Armadura 2D de segundo orden corrotacional.*
- *[Pendiente] Armadura 2D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Armadura 3D de segundo orden corrotacional.*
- *[Pendiente] Armadura 3D de segundo orden corrotacional con material no lineal.*

- *[Pendiente] Cable 2D corrotacional con ley constitutiva no lineal a tracción (relajación, fluencia, daño tensional).*
- *[Pendiente] Cable 3D corrotacional con ley constitutiva no lineal a tracción.*
- *[Pendiente] Cable 2D de segundo orden corrotacional.*
- *[Pendiente] Cable 2D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Cable 3D de segundo orden corrotacional.*
- *[Pendiente] Cable 3D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Marco 3D Euler-Bernoulli corrotacional con grandes rotaciones.*
- *[Pendiente] Marco 3D Euler-Bernoulli corrotacional con material no lineal.*
- *[Pendiente] Marco 3D Timoshenko con deformación por cortante.*
- *[Pendiente] Marco 3D Timoshenko con material no lineal.*
- *[Pendiente] Marco 2D Timoshenko corrotacional.*
- *[Pendiente] Marco 2D Timoshenko corrotacional con material no lineal.*
- *[Pendiente] Marco 3D Timoshenko corrotacional.*
- *[Pendiente] Marco 3D Timoshenko corrotacional con material no lineal.*
- *[Pendiente] Marco de segundo orden (interpolación cuadrática) en cualquiera de las formulaciones anteriores.*
- *[Pendiente] Plasticidad distribuida fibra a fibra en marcos 2D y 3D, con discretización transversal de la sección.*
- *[Pendiente] Marco con sección variable a lo largo del eje (sección no prismática).*
- *[Pendiente] Sólido 2D con no linealidad geométrica (formulación lagrangiana actualizada o total).*
- *[Pendiente] Tetraedro lineal Tet4 (cuatro nodos), primer orden.*
- *[Pendiente] Tetraedro cuadrático Tet10 (diez nodos), segundo orden.*
- *[Pendiente] Hexaedro trilineal Hex8 (ocho nodos), primer orden.*
- *[Pendiente] Hexaedro serendípito Hex20 (veinte nodos), segundo orden.*
- *[Pendiente] Hexaedro lagrangiano Hex27 (veintisiete nodos), segundo orden.*
- *[Pendiente] Elementos de cáscara y lámina (formulación de Mindlin-Reissner para cáscaras gruesas; formulación de Kirchhoff-Love para cáscaras delgadas).*
- *[Pendiente] Elementos mixtos desplazamiento-presión (Q1-P0, Taylor-Hood) para problemas casi incompresibles.*

- *[Pendiente]* Elementos de interfaz para modelado de fricción y contacto cohesivo entre superficies.
- *[Pendiente]* Newton-Raphson con búsqueda lineal (*line-search*) para mejorar la robustez ante no convergencia. **Parcialmente entregado por ADR 0011** — *line search* disponible como *opt-in* (*line_search=True*); default *False* por la justificación bibliográfica de Borst & Sluys 1999.
- *[Pendiente]* Solver cuasi-Newton tipo BFGS para reducir el coste de actualización de la matriz tangente.
- *[Pendiente]* Generalized- α , Bossak- α — esquemas adicionales de la familia con disipación numérica.
- *[Pendiente]* Solver de relajación dinámica para búsqueda de configuraciones de equilibrio en estructuras flexibles.
- *[Pendiente]* Excitación sísmica multi-direccional simultánea (CQC3, regla 100/30/30) y multi-support seismic excitation.
- *[Pendiente]* Δt adaptativo para transitorios.
- *[Pendiente]* **LDLTSolver** — factorización LDL^T para el caso simétrico no definido positivo, reservada para la fase 2 del **ADR 0003**.
- *[Pendiente]* Algoritmos algebraicos directos paralelos (Pardiso, MUMPS) para problemas de gran tamaño.
- *[Pendiente]* Algoritmos algebraicos iterativos con preconditionamiento (gradiente conjugado preconditionado, GMRES, multimalla algebraica).
- *[Pendiente]* Problema térmico estacionario lineal y no lineal (conductividad dependiente de la temperatura).
- *[Pendiente]* Problema térmico transitorio con esquemas de integración temporal (Crank-Nicolson, theta-método).
- *[Pendiente]* Acoplamiento termo-mecánico desacoplado (*staggered*) para casos en que la influencia mecánica sobre el campo térmico sea despreciable.
- *[Pendiente]* Acoplamiento termo-mecánico monolítico para problemas con fuerte interacción bidireccional.
- *[Pendiente]* Excitación sísmica multi-direccional simultánea (CQC3, 100/30/30) y multi-support para diferencias de movimiento entre apoyos.

Decisiones de arquitectura (ADR)

- *[Pendiente]* ADR para la introducción del problema térmico, incluyendo la generalización del concepto de *DOF* a magnitudes escalares no mecánicas.
- *[Pendiente]* ADR para el acoplamiento termo-mecánico, incluyendo la elección entre estrategia desacoplada y monolítica.

Evolución de la arquitectura

- *[Pendiente] Implementación de la fase 5 — problema térmico.*
- *[Pendiente] Implementación de la fase 6 — acoplamiento termo-mecánico.*
- *[Pendiente] ...*

Solidum FEM — Manual de Arquitectura

Sigla del manual: FF-MA

Compilado el 19 de mayo de 2026, 18:46

Commit Git: 71e4602 (con cambios sin confirmar)

*Documento generado automáticamente desde los archivos fuente en
manuals/sources/architecture/ mediante python
manuals/build_architecture_manual.py. No editar directamente el PDF:
cualquier corrección debe realizarse sobre el archivo fuente correspondiente y
regenerar el manual.*